# AI-BASED INTELLIGENT MULTI-DAY ITINERARY GENERATION FOR SRI LANKA TOURISM USING RANDOM FOREST AND STRUCTURED DESTINATION CATALOGUES

**INDIVIDUAL THESIS**

Wanniarachchi P.M.R

(IT22225092)

Individual component: AI Itinerary Generator

Group ID: R26-IT-152

Report submitted in partial fulfilment of the requirements for the

Bachelor of Science (Honours) in Information Technology

Department of Information Technology

Sri Lanka Institute of Information Technology

Sri Lanka

October 2026

# DECLARATION AND SUPERVISOR CERTIFICATION

**Candidate declaration.** I declare that this thesis is my own work and does not incorporate, without acknowledgement, material previously submitted for a degree or diploma at any university or institute of higher learning. To the best of my knowledge, it contains no material previously published or written by another person except where acknowledgement is made in the text. I grant the Sri Lanka Institute of Information Technology the non-exclusive right to reproduce and distribute this thesis, in whole or in part, in print, electronic, or other media, while retaining the right to use the content in future work.

**Academic integrity and AI disclosure.** I have disclosed all use of generative AI and other automated tools in accordance with the module policy. I remain responsible for the accuracy, originality, citations, code, data, analyses, and conclusions in this thesis.

Student name: Wanniarachchi P.M.R

Student ID: IT22225092

Signature: _____ Date: _____

**Supervisor certification.** The above candidate carried out this research for the bachelor's degree thesis under my supervision.

| Role and name | Signature | Date |
|---|---|---|
| Supervisor: Mr Buddika Harshnath | | |
| Co-supervisor: Prof. Samantha Rajapaksa | | |

# ABSTRACT

Many independent travellers to Sri Lanka still build multi-day routes by hand from blogs, map applications and generic hotel search tools. These tools rarely interpret free-text travel goals against a national register of attractions, respect official visit windows, allocate nights across cities in a coherent order, or rank registered accommodation against budget, comfort tier and board type (Bed and Breakfast, Half Board or Full Board). This individual component addresses that gap with an AI Itinerary Generator that produces calendar-valid, timed, multi-day plans from curated Excel catalogues through a coordinated Random Forest pipeline.

The artefact uses `attractions.xlsx` (343 attractions with 1,096 timed visit rows) and `accomadation.xlsx` (921 registered units with BB, HB and FB price bands in LKR). From experience text, arrival and departure times, budget, comfort level and meal plan, Random Forest models bundled in `rf_bundle.joblib` predict user need (moods and categories), attraction relevance, destinations, nights and visit order, places per day, non-overlapping visit slots and one hotel per destination, while timing parameters are learned from sample itineraries in `RF_training_data.xlsx`.

A React front end and Flask REST back end expose suggestion, hotel ranking and itinerary generation, and PostgreSQL stores catalogue rows and trip input. Training runs offline with `build_training_data.py` and `train_models.py`; runtime requests only load the frozen bundle. Evaluation reports cross-validated micro-F1 for need detection and mean ROC-AUC for attraction and hotel ranking, with functional API tests and scenario checks of calendar and overlap rules. Following Design Science Research, the work contributes a reproducible, catalogue-grounded itinerary engine for Sri Lankan tourism.

**Keywords:** intelligent itinerary, random forest, natural-language travel intent, TF-IDF, Sri Lanka tourism, visit-time scheduling

# ACKNOWLEDGEMENTS

I sincerely thank my supervisor, Mr Buddika Harshnath, and co-supervisor, Prof. Samantha Rajapaksa, for their guidance on research design, evaluation and academic writing throughout this project. Their feedback on the calendar rules and on how to evaluate the models honestly shaped much of this thesis.

I acknowledge the Department of Information Technology of the Sri Lanka Institute of Information Technology for the laboratory and library resources provided. I thank the members of group R26-IT-152 for the shared platform infrastructure, and the peers who helped with catalogue validation, integration testing and feedback on the itinerary wizard. The structured attraction and accommodation data were compiled for educational research, and official statistics from the Sri Lanka Tourism Development Authority informed the problem framing. Finally, I thank my family for their patience and encouragement.

# TABLE OF CONTENTS

1 INTRODUCTION .....1

1.1 Overall project context .....1

1.2 Individual component context .....2

1.3 Problem statement .....4

1.4 Research gap .....6

1.5 Aim and objectives .....7

1.6 Research questions or hypotheses .....7

1.7 Scope limitations and assumptions .....8

1.8 Individual contribution .....8

1.9 Thesis structure .....9

2 LITERATURE REVIEW AND RELATED WORK .....10

2.1 Review method and scope .....10

2.2 Domain and theoretical foundations .....10

2.3 Related research .....12

2.4 Existing systems products and techniques .....13

2.5 Gap synthesis and conceptual framework .....14

3 RESEARCH METHODOLOGY .....16

3.1 Research design .....16

3.2 Data participants or study materials .....16

3.3 Experimental or study procedure .....20

3.4 Measures and evaluation metrics .....22

3.5 Data analysis .....23

3.6 Ethics privacy security and responsible AI .....23

3.7 Reproducibility and research artefacts .....24

4 PRODUCT AND SOLUTION DESIGN ..... 25
4.1 Requirements ..... 25
4.2 Component architecture within the complete solution ..... 26
4.3 Design alternatives and decisions ..... 28
4.4 Detailed design ..... 28
4.5 Standards and professional practice ..... 33
5 IMPLEMENTATION INTEGRATION AND TESTING ..... 34
5.1 Development environment and technology stack ..... 34
5.2 Component implementation ..... 34
5.3 Integration with the other components ..... 36
5.4 Verification and software testing ..... 37
5.5 Deployment and operation ..... 38
6 RESULTS AND VALIDATION ..... 39
6.1 Research results ..... 39
6.2 Product and technical results ..... 41
6.3 User or stakeholder validation ..... 44
6.4 Integration results ..... 44
6.5 Summary of findings ..... 44
7 DISCUSSION ..... 45
7.1 Interpretation of findings ..... 45
7.2 Comparison with literature baselines and existing solutions ..... 46
7.3 Research contribution ..... 46
7.4 Product contribution ..... 47
7.5 Limitations and threats to validity ..... 47
7.6 Implications ..... 48

8 COMMERCIALISATION AND SUSTAINABILITY**Error! Bookmark not defined.**

8.1 Value proposition and target users.....**Error! Bookmark not defined.**

8.2 Market and competitive position.....**Error! Bookmark not defined.**

8.3 Business or sustainability model.....**Error! Bookmark not defined.**

8.4 Scalability intellectual property and compliance**Error! Bookmark not defined.**

8.5 Contribution to the Sustainable Development Goals**Error! Bookmark not defined.**

9 CONCLUSION AND FUTURE DIRECTIONS.....51

9.1 Conclusion .....51

9.2 Achievement of objectives .....51

9.3 Individual research and product contributions.....52

9.4 Future directions .....52

REFERENCES .....53

GLOSSARY .....57

APPENDICES .....58

Appendix A Research instruments or extended results.....58

Appendix B Academic integrity and AI use disclosure .....59

TEMPLATE CONVENTIONS.....**Error! Bookmark not defined.**

# LIST OF FIGURES

Figure 1.1: Calendar convention of a generated itinerary.....3
Figure 2.1: Conceptual framework of the AI Itinerary Generator.....15
Figure 3.1: Number of attractions per destination (top 15).....18
Figure 3.2: Distribution of moods and categories in the attraction catalogue.....18
Figure 3.3: Distribution of visit start hours across all time-slot rows .....19
Figure 3.4: Accommodation units by category group and median nightly price by meal plan.....19
Figure 3.5: Research and model development procedure .....20
Figure 3.6: Generation of the synthetic training workbook .....21
Figure 4.1: Three-layer architecture of the AI Itinerary Generator .....27
Figure 4.2: Data model and catalogue synchronisation .....27
Figure 4.3: Model pipeline inside generate() .....29
Figure 4.4: Greedy placement of one attraction .....32
Figure 5.1: Steps of the itinerary wizard.....36
Figure 5.2: Sequence of an itinerary generation request.....37

# LIST OF TABLES

Table 1.1: Tourist planning problems in Sri Lanka and coverage by the component ..4
Table 1.2: Specific objectives and delivering artefacts .....7
Table 2.1: Summary of related research .....13
Table 2.2: Comparison of existing solutions.....14
Table 3.1: Study materials.....16
Table 3.2: Sheets of the training workbook .....21
Table 3.3: Evaluation metrics.....22
Table 3.4: Research artefacts.....24
Table 4.1: Functional requirements .....25
Table 4.2: Non-functional requirements .....26
Table 4.3: Design alternatives and decisions .....28
Table 4.4: Contents of rf_bundle.joblib .....29
Table 4.5: User journey through the AI Itinerary Generator.....33
Table 5.1: Technology stack .....34
Table 5.2: Implementation modules .....34
Table 5.3: Functional and integration test cases.....37
Table 6.1: Size of the training workbook (RF_training_data.xlsx) .....39
Table 6.2: Metrics of the final train_models.py run .....40
Table 6.3: RF pipeline compared with rule-based baselines.....41
Table 6.4: Excerpt of a generated itinerary .....42
Table 6.5: Scenario checks on generated itineraries.....43
Table 7.1: Rule-based parts of the component .....47
Table 9.1: Achievement of objectives .....51
Table A.1: Main columns of Train_Attractions .....58
Table A.2: Main columns of Train_Hotels .....58
Table A.3: Main columns of Train_Visit_Times .....58
Table B.1: Use of generative AI tools .....59

# LIST OF ABBREVIATIONS

| Abbreviation | Meaning |
|---|---|
| AI | Artificial Intelligence |
| API | Application Programming Interface |
| AUC | Area Under the ROC Curve |
| BB | Bed and Breakfast |
| CV | Cross-Validation |
| DSR | Design Science Research |
| FB | Full Board |
| FIT | Free Independent Traveller |
| FR | Functional Requirement |
| HB | Half Board |
| JSON | JavaScript Object Notation |
| LKR | Sri Lankan Rupee |
| LLM | Large Language Model |
| ML | Machine Learning |
| NFR | Non-Functional Requirement |
| OOF | Out-Of-Fold |
| POI | Point of Interest |
| REST | Representational State Transfer |
| RF | Random Forest |
| ROC | Receiver Operating Characteristic |
| SDG | Sustainable Development Goal |
| SLTDA | Sri Lanka Tourism Development Authority |
| TF-IDF | Term Frequency–Inverse Document Frequency |
| TTDP | Tourist Trip Design Problem |

| Abbreviation | Meaning |
| --- | --- |
| UAT | User Acceptance Testing |
| UI | User Interface |

# 1 INTRODUCTION

This chapter introduces the research. It first places the work within digital tourism and the group platform, then describes the individual component, the problem it addresses and the research gap. The aim, objectives and research questions are stated, followed by the scope, the author's individual contribution and the structure of the thesis.

## 1.1 Overall project context

Digital tourism platforms have become the default entry point for international travel planning. Travellers search for places, read reviews, compare accommodation and build plans on their phones long before they arrive. In Sri Lanka, visitor volumes have recovered strongly: the country recorded a record 2,362,521 tourist arrivals in 2025, an increase of 15.1% over the 2,053,465 arrivals of 2024 [1, 2]. Spending has not kept pace with arrivals. Survey-based average expenditure per tourist per day was revised from USD 171.74 for January–July 2025 to USD 148.26 for August–December 2025 [3], and the Central Bank of Sri Lanka has noted that the recovery in arrivals has outpaced the recovery in tourism earnings [1, 4]. National policy therefore emphasises higher-quality experiences, longer stays and higher value per visitor. Spreading visitors across regions is also important, because concentration on a few famous sites creates pressure that has been described in the overtourism literature [5, 6].

However, most digital tools available to tourists work at the level of an isolated point of interest or a single-night lodging search. A map application can show how to reach a waterfall, and a booking platform can show available rooms in a town, but neither implements a unified decision pipeline that starts from a traveller's stated experience goals and ends with a feasible day-by-day schedule grounded in registered inventory. The traveller is left to connect these pieces by hand.

Undergraduate group projects in information technology often combine several specialised modules behind a shared database and API layer. Within such a platform, each student delivers an individual thesis that must stand alone as a research and

engineering contribution while remaining technically compatible with the shared infrastructure. The platform developed by group R26-IT-152 follows this model: the members share a PostgreSQL database, a Flask REST back end and a React front end, and each member owns one intelligent component. This thesis documents the component responsible for intelligent, personalised, multi-day itinerary generation for Sri Lanka.

### 1.2 Individual component context

The AI Itinerary Generator is the individual component assigned to the author. Its purpose is to let a logged-in traveller enter trip constraints and natural-language intent and to output a structured itinerary consisting of airport arrival handling, hotel check-in events, timed attraction visits on the middle days of the trip and a departure day without new sightseeing. After logging in, the traveller provides six inputs before planning starts: arrival date and time, departure date and time, a free-text description of the desired experience, a nightly accommodation budget, a comfort level and a meal plan (BB, HB or FB).

Implementation centres on a Python package, `rf_itinerary`, which loads `attractions.xlsx` and `accomadation.xlsx`, builds `RF_training_data.xlsx`, trains `models/rf_bundle.joblib` and exposes `generate_itinerary.generate()` for runtime planning. The back-end service `rf_itinerary_service.py` bridges this package to the Flask application, while `places_rfc_service.py` and `accommodation_rfc_service.py` route attraction suggestion and hotel scoring through the same bundle. The React user interface guides tourists through attraction selection, accommodation choice with meal-specific price ranges and a consolidated itinerary timeline, which is enriched for presentation by `itinerary_present_service.py`.

Figure 1.1 illustrates the calendar convention used throughout this thesis. The first day of the trip is reserved for arrival, when the traveller is transferred from Bandaranaike International Airport to the hotel at the first destination. The last day is reserved for shopping, rest and departure. Sightseeing is scheduled only on the days in between, which are called *travel days*.

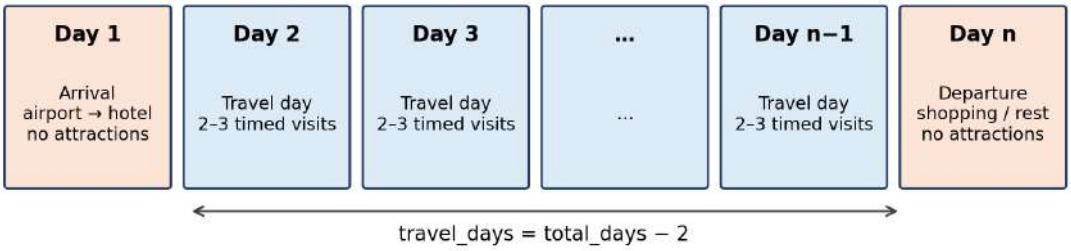



**Figure 1.1: Calendar convention of a generated itinerary**

### How the component helps the traveller

The value of the component lies in what it removes from the traveller's workload. After logging in, the traveller only describes the trip; the system then does the work that would otherwise take hours of searching across blogs, maps and booking sites:

- **Attractions for the days of the visit.** The Random Forest models read the experience text, infer the moods and categories the traveller is looking for and list the best-matching registered attractions, spread over the travel days between arrival and departure.
- **Accommodation that fits.** For each destination where the attractions are located, the hotel model suggests registered accommodation that suits the traveller's budget, comfort level and chosen meal plan (BB, HB or FB), and shows the nightly and total price range.
- **Realistic visit times.** A Random Forest also chooses the start and end time of each visit from the official visit windows, so that visits on the same day never overlap and suit the traveller's mood.
- **A ready-to-use plan.** Arrival, hotel check-ins, timed visits with activities and details, and a free last day for shopping and departure are combined into one itinerary with hotel contact details.
- **A personal saved directory.** The traveller can save shops, travel agencies and tour guides to their account and return to them later, so useful local contacts are not lost during the trip.

### 1.3 Problem statement

Independent tourists planning a trip to Sri Lanka must resolve four coupled decisions at the same time: which attractions match their interests; how many nights to spend in each region and in which order; which visit time slots avoid overlap and respect opening hours; and which registered hotels fit their budget, comfort expectation and board type. These decisions depend on each other. Choosing a beach town changes which hotels are relevant; choosing three long visits on one day may make a third visit impossible; choosing Half Board changes which hotels are affordable.

Manual planning is therefore time-consuming and error-prone. Generic large-language-model chat tools can produce fluent itineraries, but they may suggest venues that do not exist, opening times that contradict official data, or hotels whose prices are invented. For a platform that wants to promote registered attractions and accommodation, such answers cannot be trusted.

Practitioner guides and official reports describe the same difficulties from the traveller's side: complex multi-region routing with underestimated mountain-road travel times [7, 8], fragmented information spread across blogs, maps and guesthouse advice [9], mismatches between hotel prices, meal plans and budgets [10], and confusion around arrival and departure days [11]. Table 1.1 lists the main planning problems reported for Sri Lanka and shows which of them this component addresses.

**Table 1.1: Tourist planning problems in Sri Lanka and coverage by the component**

| Tourist problem | Impact | How the AI Itinerary Generator responds | Coverage |
|---|---|---|---|
| Multi-day route design across regions; map distances underestimate mountain-road time [7, 8] | Rushed trips, backtracking, exhaustion | dest_clf, dest_days and dest_order_model choose cities, nights and order; drive times estimated between cities | Addressed |
| Fragmented information from blogs, maps, booking sites and messaging apps [9] | Hours of manual research, inconsistent advice | One pipeline from experience text to a timed plan with hotels | Addressed |
| Matching free-text interests to structured attraction data | Wrong or missed sights | need_model and att_model over name, category, mood, activities and details | Addressed |

| Tourist problem | Impact | How the AI Itinerary Generator responds | Coverage |
|---|---|---|---|
| Visit windows and same-day overlaps | Closed sites, skipped visits, stress | slot_model chooses the slot; scheduler blocks overlaps and keeps a gap | Addressed |
| Hotel–budget–meal mismatch among ~900 options [10] | Overspending or poor stays | hotel_model scores the full registry; BB/HB/FB minimum–maximum shown per night and per stay | Addressed |
| Too many hotel changes and overpacked days [8] | Fatigue, higher transfer cost | Learned nights per city and ppd_model (2 or 3 visits per day) | Addressed |
| Sightseeing planned on arrival or departure day [11] | Missed transfers, tight departures | Day 1 arrival and check-in only; last day shopping and departure only | Addressed |
| Planning friction lowering the value of each trip while arrivals grow faster than earnings [1, 4] | Less value per visitor | Faster, registry-grounded plans give more time at chosen sites and hotels | Indirectly |
| Keeping track of useful shops, travel agencies and tour guides found while planning [9] | Contacts lost across chats and notes | Saved directory: the user saves shops, agencies and guides to their account for later reference | Addressed |
| Self-arranging trains, drivers and tickets [12, 13] | Planning load, sold-out trains | Reduces planning load for attractions, hotels and timetable; no booking | Partly |
| Wrong coast or month for the monsoon [14, 15] | Rain, poor beach days | Dates and text drive the plan; no live weather engine | Partly |
| Crowding at popular sites [13] | Queues | Timed slots from the catalogue; no live crowd prediction | Partly |
| Hallucinated places from generic AI chat planners | Visiting non-existent places | Outputs only registered catalogue entities | Addressed |

The core problem addressed in this research is the absence of a reproducible, data-grounded, machine-learning-driven itinerary engine that:

- maps free-text experience descriptions to structured attraction records;
- coordinates destination and duration decisions with attraction relevance scores;
- selects visit slots from the multiple timed rows recorded for each attraction;
- scores the full accommodation registry for each destination using budget, comfort and meal plan; and
- enforces inbound tour calendar conventions on the first and the final day.

### 1.4 Research gap

A review of research and existing products (Chapter 2) revealed five gaps that this component addresses.

**Gap 1 – Natural language to multi-field catalogue mapping.** Users express goals in prose, but national inventory stores attractions across separate fields: name, category, mood, activities and details. Few systems learn relevance labels tied to those fields; most rely on brittle keyword rules or require users to tick predefined categories [16].

**Gap 2 – Unified Random Forest pipeline.** Attraction ranking, routing, visit-time choice and hotel selection are usually treated as separate heuristics or separate products. This component trains one deployable bundle in which all models share the same trip context and text vectors.

**Gap 3 – Visit-time overlap and calendar rules.** Attractions in the catalogue expose several visit windows with different moods and ratings. A planner must learn which slot to choose, while a deterministic scheduler must still guarantee that visits do not overlap and that arrival and departure days follow policy. Optimisation-based trip planners handle time windows [17, 18] but assume that interest scores are already known.

**Gap 4 – Full-registry hotel fit with meal-specific LKR bands.** The accommodation registry contains 921 units with separate BB, HB and FB minimum and maximum prices. Runtime scoring must consider budget, comfort, distance, rating and rating count for every candidate rather than a hard-coded short list.

**Gap 5 – Reproducible academic artefacts.** Examiners and future maintainers need documented training sheets, scripts and metrics. Many prototypes cannot be retrained when the data change. This component supplies `RF_training_data.xlsx`, `build_training_data.py`, `train_models.py` and `RETRAIN_ALL_RF.bat` for replication.

## 1.5 Aim and objectives

**Aim.** To design, implement and evaluate an AI Itinerary Generator that uses Random Forest models and TF-IDF text features on curated Sri Lankan catalogues to produce calendar-valid, timed, multi-day itineraries accessible through a web application and REST APIs.

**Objectives.** The aim is achieved through seven specific objectives, which are summarised in Table 1.2 together with the main artefact that delivers each objective.

**Table 1.2: Specific objectives and delivering artefacts**

| ID | Objective | Main artefact |
|---|---|---|
| O1 | Maintain and synchronise `attractions.xlsx` and `accomadation.xlsx` into the RF package and PostgreSQL. | `seed_data.py`, migration 008 |
| O2 | Construct a training workbook with labelled trips, attractions, hotels, visit times, places per day, destinations, sample itineraries and geo coordinates. | `build_training_data.py`, `RF_training_data.xlsx` |
| O3 | Train a bundle with need, attraction, destination, slot, hotel and places-per-day models, and persist timing parameters and the daily visiting window. | `train_models.py`, `rf_bundle.joblib` |
| O4 | Implement runtime generation and back-end integration for suggestion, hotel ranking and itinerary building. | `generate_itinerary.py`, Flask services |
| O5 | Enforce the business calendar rules of the first and the final day. | `generate_itinerary.py`, `trip_calendar_rules` |
| O6 | Evaluate the models with cross-validated AUC and F1 and the system with functional API tests against keyword and even-split baselines. | Training log, test cases |
| O7 | Document reproducibility and operational procedures for retraining when catalogues change. | `RETRAIN_ALL_RF.bat`, this thesis |

## 1.6 Research questions or hypotheses

The research is guided by three research questions and two hypotheses.

- **RQ1** – Can Random Forest classifiers trained on synthetic trip labels achieve useful discrimination (ROC-AUC) for attraction and hotel relevance compared with keyword baselines?
- **RQ2** – Does a multi-model bundle improve end-to-end itinerary coherence compared with rule-only day allocation?

- **RQ3** – Can visit-slot selection be learned from labelled alternatives while a greedy scheduler guarantees that visits do not overlap?
- **H1** – TF-IDF similarity features combined with need probabilities provide sufficient signal for attraction relevance at the current catalogue size.
- **H2** – Group-wise cross-validation by `trip_id` yields stable hotel-ranking AUC despite sparse labels per destination.

### 1.7 Scope limitations and assumptions

**Scope.** The component covers on-shore leisure routing for the registered attractions and accommodation units in the project catalogues. Trips start and end at Bandaranaike International Airport. Middle-day sightseeing is scheduled within a daily visiting window learned from the training samples. Visa and travel authorisation, international flights, train and driver booking, live weather and monsoon routing, real-time crowd prediction, personal safety and payments are outside the scope; these correspond to the rows marked *Partly* in Table 1.1.

**Limitations.** Training messages are synthetically generated from catalogue text, so real user wording may differ. Hotel coordinates are approximated at Local Authority level, with a deterministic geo-hash fallback for unmapped authorities. Road travel uses straight-line distance multiplied by a road factor and a fixed average speed, not live traffic. The HB and FB price bands are maintained in the catalogue and are only as accurate as the data supplied by the team.

**Assumptions.** The catalogues are accurate; the visit start and end times in `attractions.xlsx` are operable; and users provide truthful dates and budgets.

### 1.8 Individual contribution

The author designed the Random Forest feature-engineering scheme, implemented `build_training_data.py` and `train_models.py`, implemented the runtime generator and its scheduling logic, integrated `rf_bundle.joblib` into the Flask services, added the meal-plan price columns to the catalogue and database, and delivered the Attractions, Accommodation and Itinerary front-end flows. Shared group infrastructure, such as authentication and the base application shell, is used but not claimed as individual work. The machine-learning design, the training-label logic

and the scheduling code paths described in Chapters 3 to 5 are the author's individual deliverables.

### 1.9 Thesis structure

Chapter 2 reviews the literature and existing systems and presents the conceptual framework. Chapter 3 explains the research methodology and data. Chapter 4 presents the requirements and the design of the component, and Chapter 5 its implementation, integration and testing. Chapter 6 reports the results, which are discussed in Chapter 7. Chapter 8 considers commercialisation and sustainability, and Chapter 9 concludes the thesis with future directions. The references, glossary and appendices follow.

# 2 LITERATURE REVIEW AND RELATED WORK

This chapter reviews the theoretical foundations and related research that informed the component, compares existing products, and closes with a synthesis of the gap and the conceptual framework.

## 2.1 Review method and scope

A structured review was carried out using Google Scholar, IEEE Xplore, ScienceDirect and MDPI journals, focusing on publications from 2020 to 2025 for tourism applications and including earlier seminal works for methods. The search terms combined *tourism recommendation*, *itinerary*, *tourist trip design*, *Random Forest*, *visit prediction*, *overtourism*, *TF-IDF* and *natural-language intent*. Empirical studies applying machine learning to itinerary planning, point-of-interest ranking or visitor prediction were included; purely policy-oriented papers were excluded except where they explained the need for demand-side digital tools. Ten primary tourism sources anchor Section 2.3, and methodological sources on recommender systems, text representation and evaluation are cited where relevant.

## 2.2 Domain and theoretical foundations

### Tourism recommender systems

Recommender systems estimate how useful items are to a user and present the most useful ones [19]. Two main families exist. *Collaborative filtering* exploits similarities between users' past ratings, while *content-based* recommendation compares item descriptions with a profile of the user's interests [20]. Hybrid systems combine both to reduce their individual weaknesses [21]. In tourism, items are attractions, hotels or complete trips, and the decision depends strongly on context such as time, location and budget [16]. Because a new platform has no interaction history, content-based approaches are particularly suitable: they work for new users and new items, avoiding the cold-start problem [22]. The attraction catalogue used here is rich in metadata (category, mood, activities and details), which favours a content-based or hybrid design.

### Text representation with TF-IDF

To compare a traveller's message with attraction descriptions, text must be represented numerically. The vector-space model with TF-IDF weighting gives a term a high weight when it is frequent in a document but rare in the collection [23]. The cosine of the angle between two TF-IDF vectors measures their similarity independently of text length. TF-IDF is fast, transparent and needs no large training corpus. Transformer models capture meaning beyond shared words [24] but require more data and computation, and their behaviour is harder to explain to examiners and catalogue maintainers. For a catalogue of a few hundred attractions, TF-IDF with cosine similarity supplies a strong and interpretable set of `sim_*` features.

### Random Forests

A Random Forest is an ensemble of decision trees, each trained on a bootstrap sample of the data and considering a random subset of features at each split; predictions are averaged over the trees [25]. Averaging reduces the variance of individual trees, so forests are accurate and robust with little tuning. They handle numeric and categorical features without scaling, capture non-linear interactions such as the joint effect of budget, distance and rating on hotel suitability, and provide feature importances that support explanation [26]. Tree ensembles also remain highly competitive on small and medium tabular datasets compared with deep learning [27]. These properties match the mixed text and tabular features of this component.

### Trip design and scheduling

Selecting which places to visit within a limited time is formalised as the Tourist Trip Design Problem (TTDP), usually modelled as a variant of the orienteering problem in which each location has a score and the route must respect a time budget [28, 29]. Extensions add time windows, multiple days and hotel selection [18]. These models produce excellent routes once scores are known, which is exactly where learned relevance models can contribute.

### Learning without real trip logs

When ground-truth labels are not available, *weak supervision* creates training labels from heuristics or domain knowledge and trains a model to generalise from them [30]. This is the situation of a new tourism platform: there are no historical

itineraries, but the catalogue itself contains enough structure to generate realistic synthetic trips. When several models depend on each other's predictions, out-of-fold predictions must be used during training to avoid information leakage [31].

### 2.3 Related research

**Visitor prediction with machine learning.** Bravo et al. [32] applied linear regression, k-nearest-neighbour regression, decision trees and Random Forests to predict visitors to the tourist attractions of the Moche Route in Peru, following a four-stage process of data collection, model analysis, model development and evaluation. Their work shows that tree-based ensembles are practical for tourism data that combine temporal and contextual attributes. Such studies predict aggregate demand rather than recommending items to an individual, but they support the choice of Random Forests for tourism problems of moderate data size.

**Overtourism and demand-side tools.** Mihalič [5] conceptualised overtourism from a sustainability perspective, arguing that tourism becomes unsustainable when its impacts exceed what residents and visitors can accept. Santos-Rojo et al. [6] carried out a bibliometric study of overtourism research from 2018 to 2021 and showed how closely the topic is now linked to sustainability and to the management of visitor flows. For this research, these works imply that an itinerary generator should not only satisfy the individual traveller but can also distribute visits across a wider set of registered attractions.

**Random Forests versus deep models.** Breiman's original work [25] established the Random Forest method. Grinsztajn et al. [27] benchmarked tree-based models against deep learning on tabular data and found that tree ensembles remain superior on medium-sized datasets, which matches the scale of this component's catalogue of a few hundred attractions and about nine hundred hotels. Feature importances of forests can also be inspected to explain decisions [26], which is valuable when results must be defended to examiners and catalogue owners.

**Personalised itinerary planning.** Vansteenwegen et al. [17] built the City Trip Planner, which collects interest ratings for predefined categories and solves a multi-day orienteering problem with opening hours. Lim et al. [33] personalised trip

recommendation by estimating interest from historical photo traces and adjusting visit durations accordingly. Borràs et al. [16] surveyed intelligent tourism recommender systems and observed that few systems integrate preference elicitation, recommendation and scheduling. Learning-to-rank research [34] shows that supervised models combining many weak relevance signals usually outperform single hand-tuned scores. Table 2.1 summarises these works.

**Table 2.1: Summary of related research**

| Study | Focus | Method | Relevance to this work |
|---|---|---|---|
| Bravo et al. [32] | Visitor prediction, Moche Route (Peru) | Regression, KNN, decision tree, RF | Supports RF on tourism data |
| Mihalič [5]; Santos-Rojo et al. [6] | Overtourism and sustainability | Conceptual / bibliometric review | Need to spread visits across registered places |
| Grinsztajn et al. [27] | Tabular learning benchmark | Trees vs deep learning | Trees preferred at medium data size |
| Vansteenwegen et al. [17] | City trip planning | Orienteering heuristics | Time windows; needs interest scores |
| Lim et al. [33] | Personalised trips | Interest from history | Needs user history |
| **This work** | **Free text → timed multi-day plan + hotels** | **Six RF models + scheduler** | **No user history needed** |

## 2.4 Existing systems products and techniques

Practitioner sources confirm that independent travellers in Sri Lanka still combine many tools by hand: guesthouse owners often act as informal travel agents [9], train seats must be booked within a short reservation window [12], and travellers are advised to avoid changing hotels every night and to start early at popular sites [8, 13]. These sources are secondary, but they describe the planning friction that motivates this work.

Several kinds of products support tourists today. **Google Maps** shows places, opening hours and directions but leaves the selection and ordering of places to the user. **TripAdvisor** aggregates reviews and lets users save places to trips. **Booking platforms** rank accommodation by price, rating and distance and support filters such

as breakfast included, but do not link the hotel to the attractions the traveller wants to see. **Generic AI planners and chatbots** generate fluent itineraries from prompts but are not grounded in a national register and may produce non-existent venues or wrong times. The **SLTDA** publishes official information and registered accommodation for Sri Lanka, and **manual spreadsheets** remain common among independent travellers and small agencies.

Table 2.2 compares these solutions against the four criteria that define this research: mood-aware relevance from free text, multi-day night allocation, timed slots from a national register, and hotel choice with BB, HB and FB price bands.

**Table 2.2: Comparison of existing solutions**

| Solution | Mood-aware relevance from text | Multi-day nights | Timed slots from register | BB/HB/FB hotel bands |
|---|---|---|---|---|
| Google Maps | No | No | Opening hours only | No |
| TripAdvisor | Partial (reviews) | No | No | No |
| Booking platforms | No | No | No | Filters only |
| Generic AI planners | Yes (ungrounded) | Yes (ungrounded) | Not verified | Not verified |
| SLTDA app / website | No | No | No | Registry only |
| Manual spreadsheets | Manual | Manual | Manual | Manual |
| **AI Itinerary Generator** | **Yes (RF)** | **Yes (RF)** | **Yes (catalogue)** | **Yes (catalogue)** |

## 2.5 Gap synthesis and conceptual framework

The review shows that tourism machine-learning studies mostly predict aggregate demand, trip-design research assumes known interest scores, recommender systems usually need interaction histories, and commercial products leave the integration of attractions, timing and accommodation to the traveller. No reviewed solution maps free text to a national attraction register, allocates nights across cities, chooses timed slots and selects meal-plan-aware hotels in one reproducible pipeline.

Figure 2.1 presents the conceptual framework derived from this synthesis. The user message is converted into need probabilities by a TF-IDF need model; the attraction model scores every catalogue entry; destination features feed the destination selection, duration and order models, producing a day-to-destination map; the slot model and scheduler place visits on travel days; and the hotel model selects one unit per destination. Deterministic layers enforce `travel_days = total_days − 2`, the overlap check and road-distance estimates.

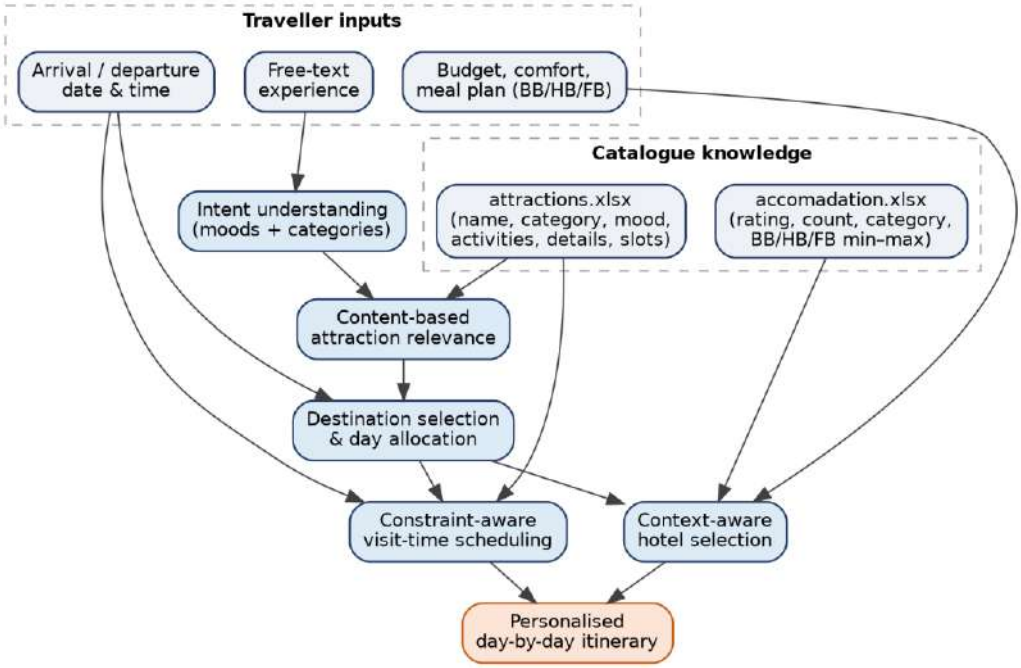



**Figure 2.1: Conceptual framework of the AI Itinerary Generator**

In this framework, the curated catalogues play two roles: they are the source of candidate attractions and hotels at runtime, and they are the source of the synthetic training labels from which the models learn. This dual role is what makes the component reproducible from the Excel files alone.

# 3 RESEARCH METHODOLOGY

This chapter describes how the research was conducted. It explains the research design, profiles the data, describes the procedure used to generate training data and train the models, defines the evaluation metrics and analysis, and discusses ethics and reproducibility.

## 3.1 Research design

The research follows Design Science Research (DSR), in which knowledge is produced by building and evaluating an artefact that solves an identified problem [35]. The DSR cycle was applied as follows: the *problem* was identified from tourist planning practice and the literature (Chapters 1 and 2); the *artefact* is the Random Forest pipeline together with the web module; the *demonstration* is the generation of itineraries in the integrated platform; and the *evaluation* combines quantitative model metrics with functional and scenario testing.

Development was iterative. An early prototype used keyword matching and fixed rules; these were progressively replaced by learned models while the itinerary format and calendar rules were kept stable. After every change to the catalogues or the label logic the models were retrained, and edge cases in scheduling (for example late arrivals, short trips and destinations with few attractions) were fixed in successive iterations.

## 3.2 Data participants or study materials

The study materials are two curated Excel catalogues and a training workbook derived from them (Table 3.1). Training required no human participants. Live trips created in the application are stored in PostgreSQL under the user's consent, and an optional user acceptance test with a small number of participants is described in Section 6.3.

**Table 3.1: Study materials**

| Material | Sheets / key fields | Size |
|---|---|---|

| Material | Sheets / key fields | Size |
|---|---|---|
| attractions.xlsx | All Data: Attraction Name, Category, Destination, Visit Start, Visit End, Visit_Duration, Mood, Rating (1–5), Activities, Details, Best Months; Destination Alias; Best Months | 1,096 time-slot rows; 343 attractions; 26 destinations |
| accomadation.xlsx | Accomadation, Ratings (out of 10), Rating count, Category, Rooms, Address, Local Authority, Web, Email, Tel, Mobile, BB/HB/FB Min/Max (LKR) | 921 units; 890 usable after load_hotels(); 163 Local Authorities |
| RF_training_data.xlsx | Trips, Train_Attractions, Train_Destinations, Train_PPD, Train_Visit_Times, Train_Hotels, Train_Itinerary_Samples, Geo | 171 synthetic trips; 8 sheets (Table 6.1) |
| PostgreSQL | Attraction, Accommodation, UserTripInput | Seeded from the catalogues |

### Attraction catalogue

Each attraction is recorded in one or more rows, one per visit slot. A slot has its own start and end time, duration, mood and rating; for example, a temple may offer a *Peaceful* dawn slot and a *Curious* mid-morning slot. On average an attraction has 3.2 slots (median 2, maximum 19). Destination names were normalised through the Destination Alias sheet so that spelling variants do not split a city. Figure 3.1 shows the distribution of attractions over the fifteen largest destinations. The catalogue is concentrated in Colombo, Ampara, Kurunegala, Matara and Galle, while several well-known areas such as Kandy, Anuradhapura and Trincomalee have only two or three entries; this imbalance influences destination selection and is discussed in Chapter 7.

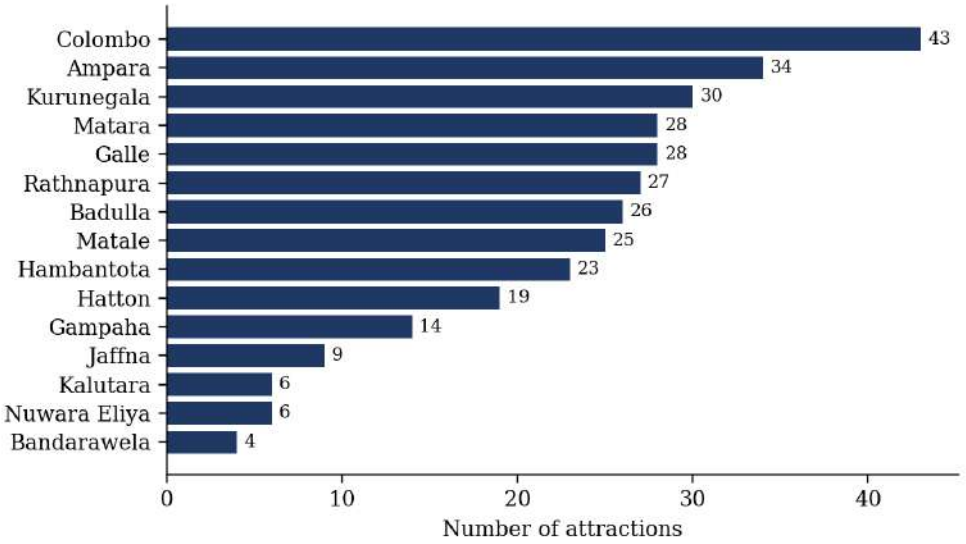



**Figure 3.1: Number of attractions per destination (top 15)**

Ten moods and six categories are used (Figure 3.2). *Curious*, *Explore* and *Peaceful* are the most frequent moods, while *Happy* and *Authentic* are rare. *Heritage* and *Scenic* dominate the categories, and *Thrills* has only seven attractions. Ratings are high overall (mean 4.37, median 4.49 on a five-point scale), so rating alone discriminates poorly between attractions.

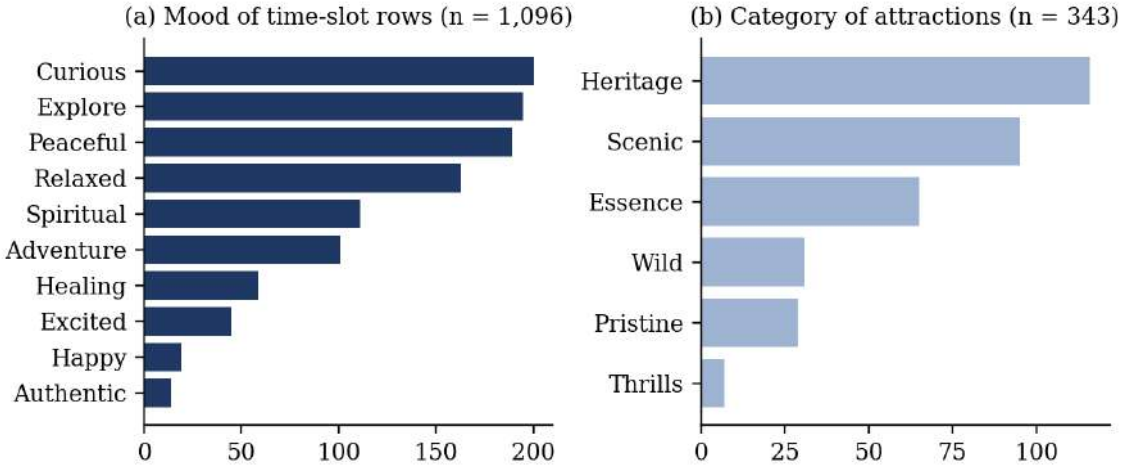



**Figure 3.2: Distribution of moods and categories in the attraction catalogue**

Visit slots cluster in two periods (Figure 3.3): early morning between 06:00 and 10:00, and late afternoon between 15:00 and 18:00, which reflects the hot midday climate. Durations range from 15 minutes to 12 hours, with a median of 90 minutes.

A few slots start very early (for example pre-dawn climbs for sunrise views) or end late, which explains why the visiting window learned from the samples is wide.

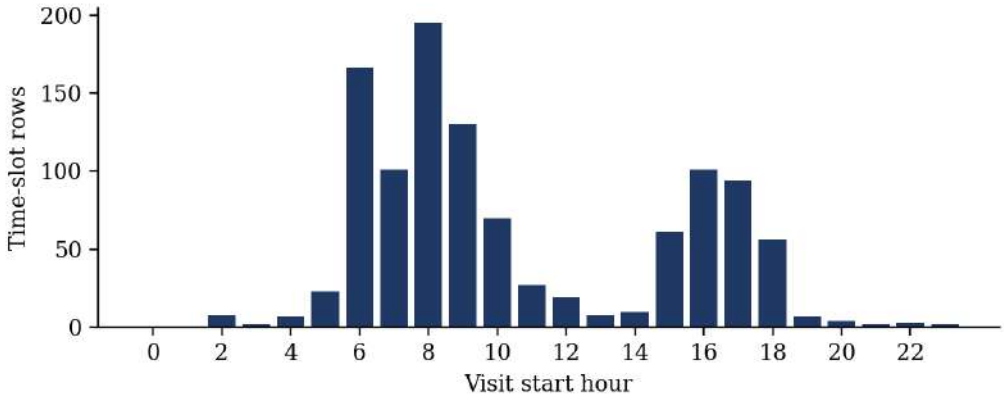



**Figure 3.3: Distribution of visit start hours across all time-slot rows**

### Accommodation catalogue

The accommodation registry follows the SLTDA classification. Guest houses form the majority (526 units), followed by bungalows (191), star-class hotels (86), unclassified hotels (55) and boutique properties (53) (Figure 3.4). Ratings are given out of ten (mean 8.68) with highly variable rating counts (median 142, maximum 20,928). Each unit has a minimum and maximum nightly price for Bed and Breakfast, Half Board and Full Board; the median BB band is LKR 11,000–16,000 per night. Units without a published price or a resolvable location are excluded by `load_hotels()`, leaving 890 units for recommendation.

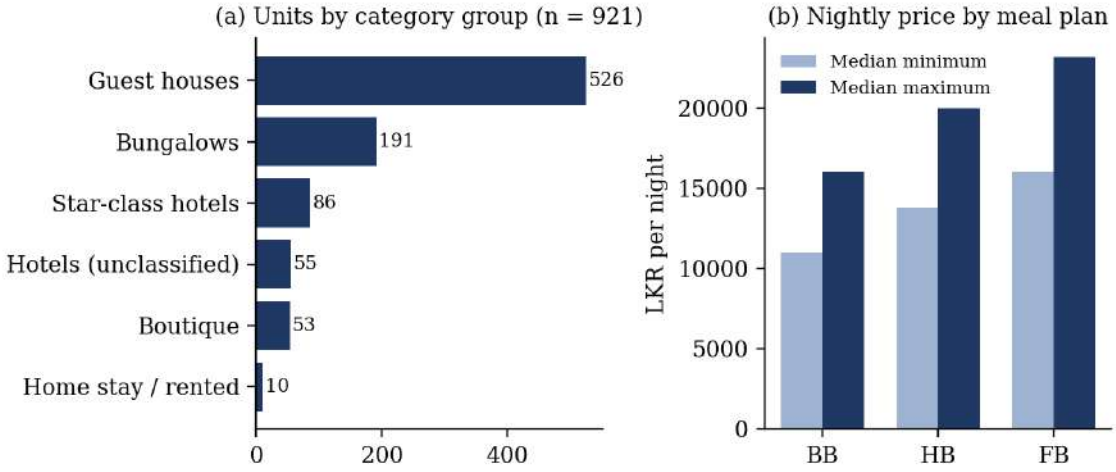



**Figure 3.4: Accommodation units by category group and median nightly price by meal plan**

### 3.3 Experimental or study procedure

Figure 3.5 summarises the procedure followed to prepare data, train the models and evaluate the component.

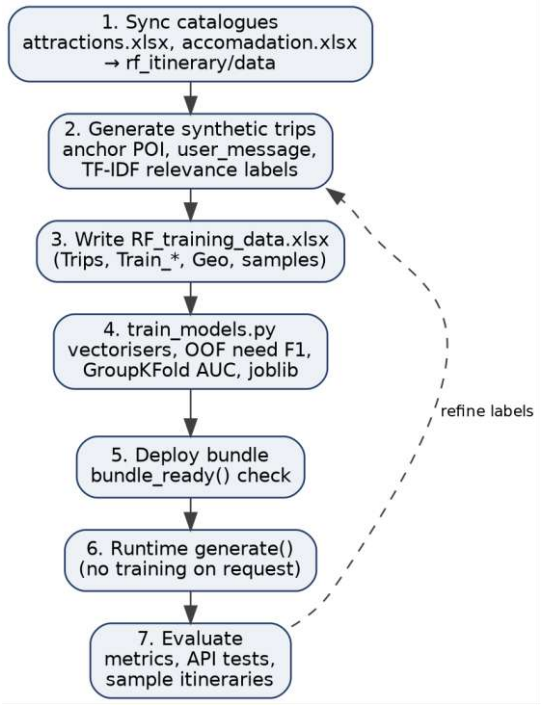



**Figure 3.5: Research and model development procedure**

#### Step 1 – Catalogue synchronisation

The two Excel files are copied into the `data` folder of the RF package and seeded into PostgreSQL by `seed_data.py`. Migration `008_accommodation_meal_prices.sql` adds the BB, HB and FB minimum and maximum columns to the accommodation table. Times stored as text or as Excel time values are parsed into minutes, visit windows that cross midnight are extended by 24 hours, and durations such as "1.5 h" or "45 min" are converted to minutes.

#### Step 2 – Synthetic trip generation

Because no historical itineraries exist, training examples were generated synthetically through weak supervision [30], as shown in Figure 3.6. For each synthetic trip, `build_training_data.py` chooses an *anchor* attraction and composes a `user_message` from tokens of its activities and details, so that the message expresses a realistic interest. Trip length, arrival and departure times, budget, comfort level and meal plan are sampled. TF-IDF similarity between the message and each

attraction, together with mood and category agreement, provides the relevance labels; the script then plans destinations, nights, places per day, visit slots and hotels, and writes every decision—chosen and rejected candidates alike—to the workbook. For hotels, the labelled candidates are the 25 units nearest to each destination.

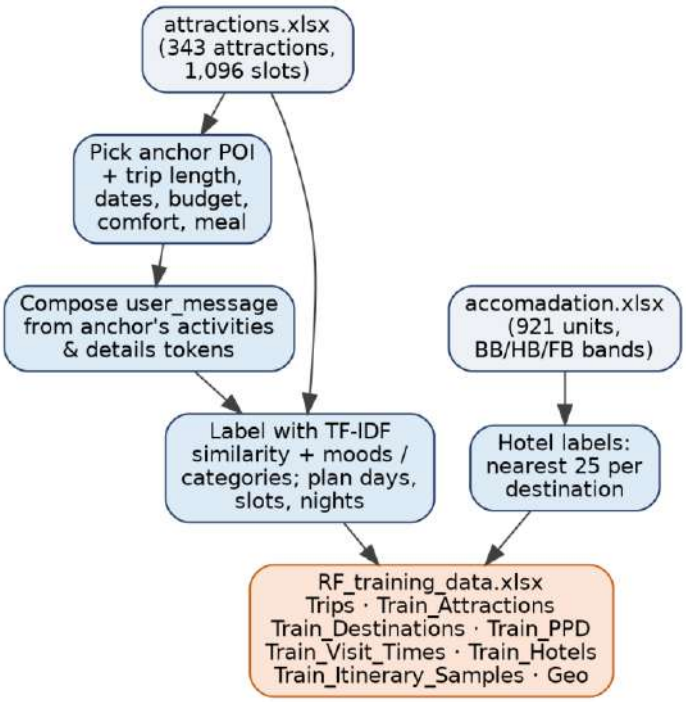



**Figure 3.6: Generation of the synthetic training workbook**

### Step 3 – Training workbook

Table 3.2 describes the sheets of `RF_training_data.xlsx`. Each sheet corresponds to one learning task, and its LABEL columns are the targets.

**Table 3.2: Sheets of the training workbook**

| Sheet | One row represents | Target(s) |
|---|---|---|
| Trips | A synthetic traveller: message, dates, budget, comfort, meal plan | Moods, categories |
| Train_Attractions | Message paired with an attraction and its fields | LABEL_relevant |
| Train_Destinations | Trip paired with a destination | LABEL_selected, LABEL_days, LABEL_order |
| Train_PPD | Travel day with its time budget and candidate durations | places_that_day (2 or 3) |
| Train_Visit_Times | Candidate slot of a planned attraction with already planned times that day | LABEL_chosen |

| Sheet | One row represents | Target(s) |
|---|---|---|
| Train_Hotels | One of the 25 nearest hotels for a destination with the user's budget, comfort and meal plan | LABEL_selected |
| Train_Itinerary_Samples | Event of a complete sample itinerary | Timing parameters (gap, checkout), day window |
| Geo | Approximate coordinates of destinations, Local Authorities and the airport | – (lookup) |

### Step 4 – Model training

`train_models.py` fits a message vectoriser (`msg_vec`) and an attraction-text vectoriser (`text_vec`), then trains the models in dependency order with scikit-learn [36]. The need model is evaluated with out-of-fold predictions, and these out-of-fold probabilities are used as features for the downstream attraction and slot models so that no model is trained on predictions made for its own rows [31]. Attraction and hotel models are evaluated with `GroupKFold` grouped by `trip_id`, so all rows of a trip are always in the same fold. Timing parameters and the daily visiting window are computed from the sample itineraries and chosen visit rows as medians and bounds; in the final run they were a 60-minute gap between visits, a 120-minute checkout offset and a visiting window of 02:00–23:00. All objects are saved in one dictionary, `rf_bundle.joblib`.

### Steps 5 to 7 – Deployment, runtime and evaluation

At start-up the back end calls `bundle_ready()`; if the bundle is missing, itinerary endpoints return HTTP 503 instead of silently falling back to rules. At runtime `generate()` only performs inference—no training happens on a request. The component is then evaluated as described in Sections 3.4 and 3.5.

## 3.4 Measures and evaluation metrics

**Table 3.3: Evaluation metrics**

| Level | Metric | Applied to |
|---|---|---|
| Need understanding | Micro-averaged F1 over 16 labels at a 0.5 threshold, out-of-fold | need_model |

| Level | Metric | Applied to |
|---|---|---|
| Ranking | Mean ROC-AUC over GroupKFold folds [37] | att_model, hotel_model |
| Product | Success of suggestion and itinerary API calls; response time (typically < 30 s including database enrichment) | Integrated system |
| Schedule validity | Overlapping visits; visits on arrival or departure day; places per travel day | Generated itineraries |
| Qualitative | Plausibility of multi-city samples checked by the author and supervisor | Generated itineraries |

ROC-AUC was chosen for ranking tasks because the generator uses model probabilities to rank candidates, and AUC measures ranking quality independently of a threshold [37]. Micro-F1 was chosen for the need model because it aggregates over all mood and category labels, which are imbalanced [38].

### 3.5 Data analysis

Metrics were computed with scikit-learn and written to the training log (Appendix A). API responses were checked against the expected JSON schema. Two baselines from the earlier rule-based implementation were used for comparison: keyword-based attraction suggestion and the even-split day plan in `trip_plan_service`, which divides travel days equally between selected cities. Sample itineraries for different trip lengths, budgets and meal plans were inspected for overlaps, calendar-rule violations, unrealistic travel and price consistency.

### 3.6 Ethics privacy security and responsible AI

Training messages are synthetic and contain no personal data. In the application, the experience text and trip details are processed server-side and stored in the user's trip record; they are not shared with third parties, in line with the principles of the Personal Data Protection Act No. 9 of 2022 [39]. Database credentials are read from environment variables. Hotel geo-hashing is documented as an approximation so that users are not misled about exact distances.

A key responsible-AI property of the design is grounding: the generator can only output attractions and hotels that exist in the registered catalogues, so it cannot invent

venues, opening times or prices in the way generic chat tools can. Every recommendation is shown with its catalogue facts (activities, details, rating, contacts and price band), which supports user trust [40]. Because the labels encode the author's planning logic, the models may reproduce its biases, for example towards highly rated or well-documented places. The use of AI writing and coding tools is disclosed in Appendix B.

### 3.7 Reproducibility and research artefacts

The full pipeline can be rebuilt from the two Excel catalogues with fixed random seeds. On Windows, `RETRAIN_ALL_RF.bat` calls `train_all_rf.py`, which runs `build_training_data.py` and `train_models.py` in sequence; the back end is then restarted. Table 3.4 lists the artefacts.

**Table 3.4: Research artefacts**

| Artefact | Purpose |
|---|---|
| `RF_training_data.xlsx` | Training workbook (Table 3.2) |
| `rf_bundle.joblib` | Trained vectorisers, models, timing parameters and day window |
| `build_training_data.py` | Synthetic trip and label generation |
| `train_models.py` | Training, cross-validation and bundle export |
| `generate_itinerary.py`, `rf_planning.py`, `common.py` | Runtime generation, destination allocation, features |
| `train_all_rf.py`, `RETRAIN_ALL_RF.bat` | One-click retraining |
| `seed_data.py`, migration 008 | Database seeding and meal-price schema |

# 4 PRODUCT AND SOLUTION DESIGN

This chapter specifies what the component must do, how it fits into the group platform, which design alternatives were considered and how the models and the generator are designed in detail.

## 4.1 Requirements

Requirements were gathered from the problem analysis, group meetings and supervisor feedback. Table 4.1 lists the functional requirements with the implementation that satisfies each, and Table 4.2 the non-functional requirements organised by ISO/IEC 25010 [41].

**Table 4.1: Functional requirements**

| ID | Requirement | Implementation |
|---|---|---|
| FR0 | Register and log in; each trip and saved item belongs to the user account. | Shared authentication |
| FR1 | Capture trip input: experience text, arrival and departure date/time, budget, comfort level, meal plan. | AttractionsPage, UserTripInput |
| FR2 | Suggest attractions ranked by the RF models. | `suggest_from_trip()` |
| FR3 | Rank the full hotel catalogue for each destination. | `ranked_hotels_for_destination()` |
| FR4 | Generate a timed multi-day plan with non-overlapping visits. | `generate()`, slot_model, scheduler |
| FR5 | Day 1: arrival and hotel only; final day: shopping and departure only. | `generate_itinerary.py` |
| FR6 | Show meal-plan minimum–maximum price per night and for the stay in LKR. | `hotel_prices_for_meal`, AccommodationPage |
| FR7 | Persist trip input and selections through the REST API. | Flask routes, PostgreSQL |
| FR8 | Present and export the itinerary. | `itinerary_present_service`, ItineraryPage |
| FR9 | Retrain all models in one batch when catalogues change. | `RETRAIN_ALL_RF.bat` |
| FR10 | Report ML bundle status; return 503 when the bundle is missing. | `bundle_ready()` endpoint |
| FR11 | Let the user save shops, travel agencies and tour guides to a personal list and view them later. | Saved directory page and user-linked records |

**Table 4.2: Non-functional requirements**

| Quality characteristic | Requirement |
|---|---|
| Performance efficiency | A typical itinerary request completes within 30 seconds including database enrichment; the bundle is loaded once. |
| Reliability | If the bundle is missing, endpoints return HTTP 503 with a clear message rather than incorrect plans. |
| Usability | The wizard works on mobile screens; each step shows only the inputs needed for that step. |
| Maintainability | A single `RF_ROOT` path configures the package; catalogue changes need only retraining. |
| Compatibility | JSON over the shared REST API; shared PostgreSQL tables with other group components. |
| Security | Secrets in environment variables; only authenticated users access their trips. |

## 4.2 Component architecture within the complete solution

The component is organised in three layers (Figure 4.1).

The **data layer** consists of the Excel catalogues, the training workbook, the PostgreSQL tables for users, attractions, accommodation, trip input and saved items, and the trained bundle. The catalogues are the single source of truth; PostgreSQL holds a seeded copy for fast lookups and stores each user's trip input and selections.

The **intelligence layer** contains the training scripts, the runtime package (`generate_itinerary`, `rf_planning`, `common`) and the Flask services. `rf_itinerary_service` resolves the package location through `RF_ROOT`, checks `bundle_ready()` and calls `generate()`. `places_rfc_service` serves attraction suggestions, `accommodation_rfc_service` ranks hotels, and `ai_service.build_itinerary_payload` builds the final itinerary.

The **presentation layer** consists of the login screen, the React wizard with three pages for planning (attractions, accommodation and itinerary), the saved directory of shops, travel agencies and tour guides, and `itinerary_present_service`, which enriches the itinerary JSON with dates, day segments, contacts and price totals.

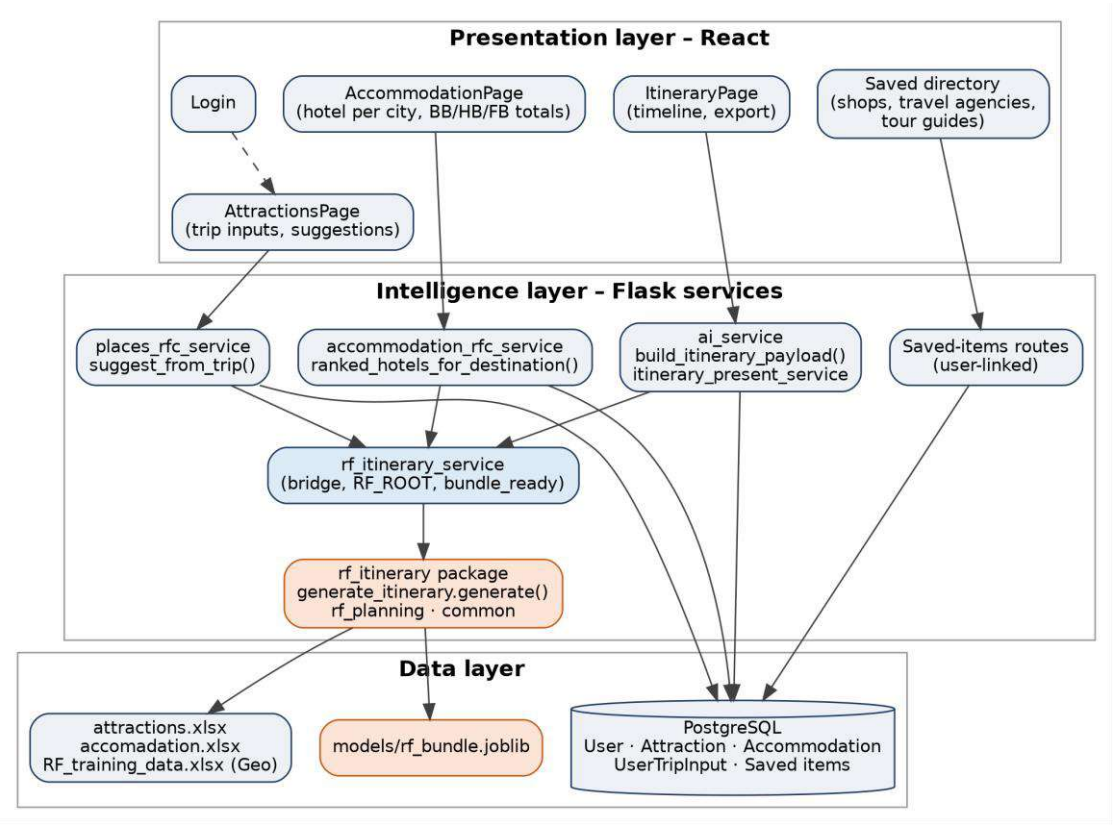



**Figure 4.1: Three-layer architecture of the AI Itinerary Generator**

Because the shared database and REST conventions are reused, the component is compatible with the other group components: the trip identifier, dates and selected places stored in `UserTripInput` are available to them through the database.

Figure 4.2 shows how the catalogues flow into both the database and the RF package.

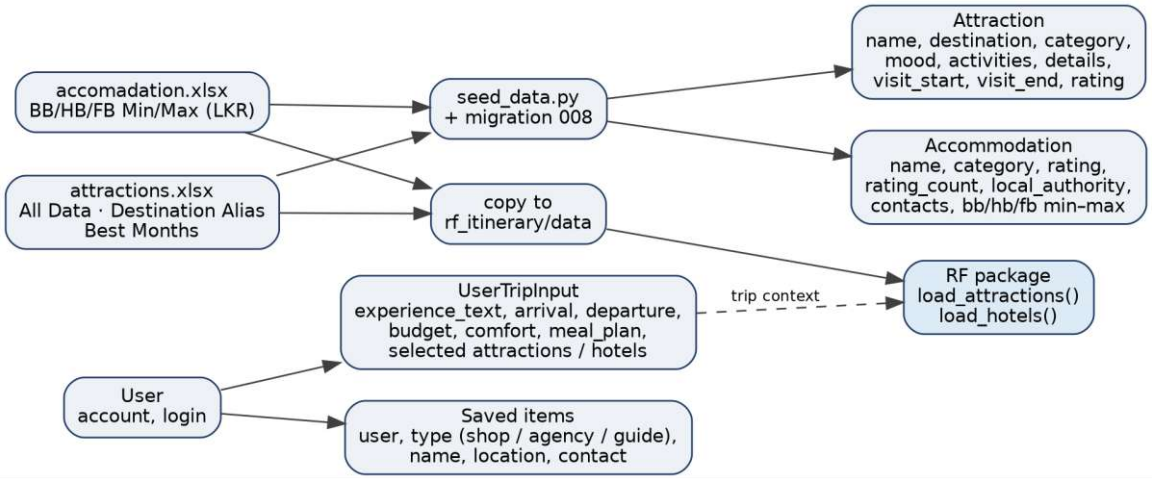



**Figure 4.2: Data model and catalogue synchronisation**

## 4.3 Design alternatives and decisions

**Table 4.3: Design alternatives and decisions**

| Decision | Alternatives | Choice and rationale |
|---|---|---|
| Core approach | Rules only; deep learning; LLM-only generation; Random Forest pipeline | RF pipeline. Rules gave insufficient personalisation; deep learning needs far more labelled data and is weaker on medium-sized tabular data [27]; an LLM alone risks hallucinated venues and times that contradict the Excel catalogue. |
| Learning algorithm | Logistic regression, gradient boosting, RF | RF handles mixed text-similarity and tabular features, needs little tuning and has precedent in tourism ML [32]. |
| Labels | Wait for real users; manual labelling; synthetic labels | Synthetic labels from the catalogue [30]: available immediately and replaceable by real feedback later. |
| Packaging | One project per task; single bundle | Single joblib bundle: one retraining command, consistent versions. |
| Hotel candidates | Short list per city; full registry | Full registry scored at runtime; distance is a feature. |
| Meal prices | Percentage uplift on one price; explicit columns | Explicit BB/HB/FB min–max columns; no price arithmetic beyond nights × band. |
| Missing bundle | Silent rule fallback; explicit error | HTTP 503, so wrong plans are never shown as ML output. |

## 4.4 Detailed design

The bundle is a single Python dictionary saved with joblib. Figure 4.3 shows the order in which its models are used, and Table 4.4 lists its contents.

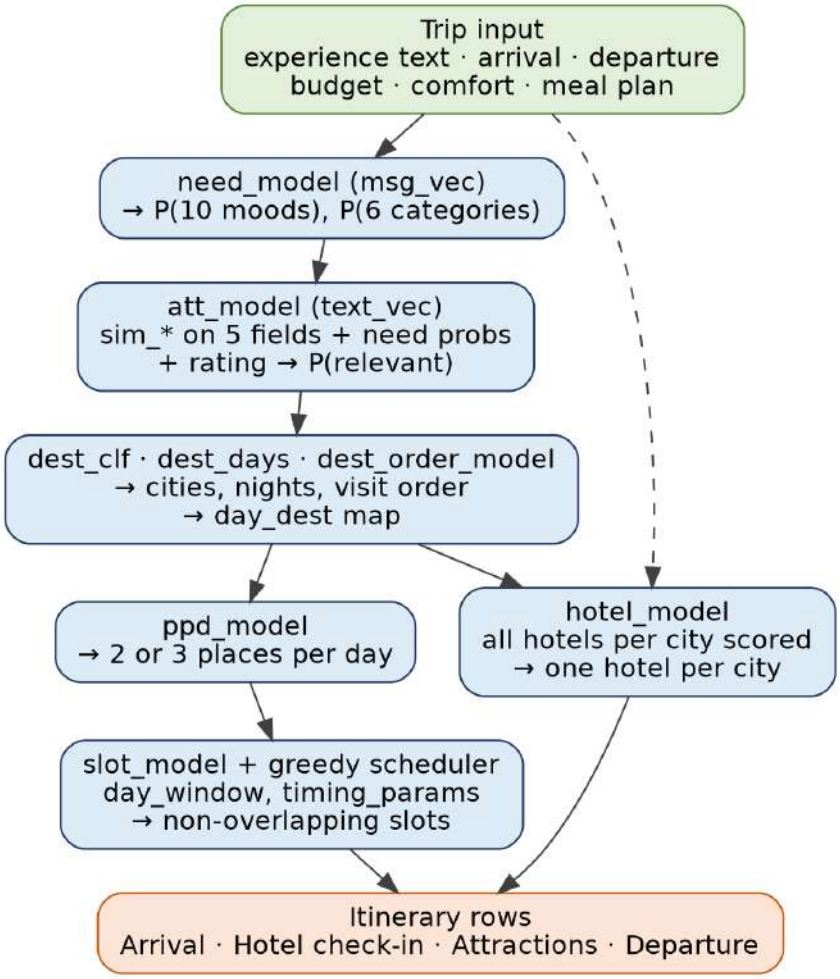



**Figure 4.3: Model pipeline inside generate()**

**Table 4.4: Contents of rf_bundle.joblib**

| Key | Type | Inputs | Output |
|---|---|---|---|
| `msg_vec` | TF-IDF vectoriser | User message | Sparse vector |
| `text_vec` | TF-IDF vectoriser | Attraction fields | Sparse vectors |
| `need_model` | Multi-output RF classifier | msg_vec features | P(10 moods), P(6 categories) |
| `att_model` | RF classifier | `sim_*` on name, category, mood, activities, details; mood and category probabilities; rating | P(relevant) |
| `dest_clf` | RF classifier | DEST_COLS | P(destination selected) |
| `dest_days` | RF regressor | DEST_COLS | Nights per destination |
| `dest_order_model` | RF regressor | DEST_COLS | Visit order |

| Key | Type | Inputs | Output |
|---|---|---|---|
| `ppd_model` | RF classifier | Day time budget, candidate durations | 2 or 3 places |
| `slot_model` | RF classifier | slot_features + overlaps_planned | P(slot chosen) |
| `hotel_model` | RF classifier | Meal-plan price vs budget, comfort, meal, distance, rating, count, category | P(hotel selected) |
| `timing_params`, `day_window` | Parameters | Sample itineraries (medians, bounds) | Gap, checkout, visiting window |

### Need model

The message is lower-cased, stop words are removed and unigrams and bigrams are weighted with TF-IDF [23]. A multi-output Random Forest predicts one probability for each of the ten moods and six categories [38]. The probabilities are passed on as continuous features rather than being turned into hard labels, so uncertain moods still contribute.

### Attraction model

For each attraction the generator computes five cosine similarities between the message and the attraction's name, category, mood, activities and details (`sim_name`, `sim_category`, `sim_mood`, `sim_activities`, `sim_details`), the mean and maximum predicted probability of the attraction's moods, the predicted probability of its category and its rating. The attraction model converts these into a relevance probability, which replaces the fixed weights and thresholds of the earlier keyword prototype.

### Destination models

Attraction probabilities are aggregated per destination into `DEST_COLS`: the mean of the top attraction probabilities, its rank, the number of attractions, the distance from the airport, the distance from the best-matching destination and the number of travel days. `dest_clf` decides which destinations to include, `dest_days` estimates the nights for each, and `dest_order_model` estimates the visiting order. Finally, the function `rf_planning.allocate_destinations()` scales the predicted nights to the

exact number of travel days and produces the `day_dest` map from day number to destination.

### Calendar mapping

The number of calendar days is computed from the arrival and departure dates inclusively, and `travel_days = total_days − 2`. Day 1 is mapped to the first destination for the hotel transfer; days 2 to $n-1$ follow `day_dest`; the last day is mapped to the last destination and contains only shopping, rest and the departure event.

### Places per day and slot scheduling

For each travel day, `ppd_model` predicts whether two or three attractions should be scheduled, based on the available time and the durations of the best candidates. Figure 4.4 shows how one attraction is placed. Attractions of a destination are taken in descending relevance; every slot of the attraction is scored by `slot_model`; slots are tried in order of probability; a slot is accepted if it lies inside the learned `day_window` and leaves at least the learned gap to other visits that day. If the best slot clashes, the attraction is moved to the same slot on the next open day. This greedy scheduler guarantees non-overlap regardless of model output.

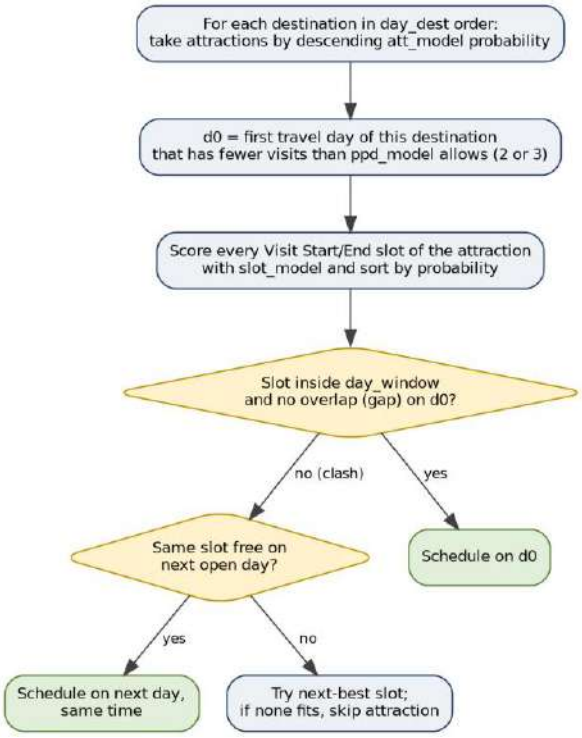



**Figure 4.4: Greedy placement of one attraction**

### Hotel model

For every destination, all priced hotels in the registry are scored. Features include the distance from the destination, rating, rating count, room count, category, the minimum, mid-point and maximum price for the chosen meal plan, the ratio of these prices to the budget, and indicators of the requested comfort level and meal plan. The highest-probability hotel is selected. During training only the 25 nearest hotels per destination were labelled, whereas at runtime all hotels are scored; the distance feature lets the model penalise distant units it did not see during training.

### Itinerary assembly

The output is an ordered list of rows: *Arrival* at the airport (CMB) with time, *Hotel check-in* at the first destination with nights, meal plan, nightly minimum–maximum price and total for the stay, timed *Attraction* rows with mood, activities and details, a new *Hotel check-in* whenever the destination changes, and on the final day *Free time / shopping* followed by *Departure*.

### User journey and how each step helps

Table 4.5 follows a traveller through the system and shows, for each step, what the traveller does, what the component does behind the scenes and how this helps.

**Table 4.5: User journey through the AI Itinerary Generator**

| Step | Traveller action | What the component does | How it helps the traveller |
|---|---|---|---|
| 1 | Logs in | Loads the user account, previous trips and saved items | Plans and contacts are kept in one place |
| 2 | Enters arrival and departure date and time, budget, experience text, comfort level and meal plan | Stores UserTripInput; need_model infers moods and categories from the text | Describes the trip in their own words instead of filling long forms |
| 3 | Reviews suggested attractions | att_model ranks all registered attractions; destination models choose cities, nights and order; ppd_model sets 2 or 3 visits per travel day | Gets attractions that match their interests, spread over the days of the visit |
| 4 | Reviews suggested accommodation | hotel_model scores every hotel near each destination using budget, comfort, BB/HB/FB price, rating, rating count and distance | Sees affordable, suitable hotels with nightly and total price ranges |
| 5 | Opens the itinerary | slot_model chooses visit start and end times; scheduler prevents overlaps; calendar rules applied | Receives a realistic, timed, day-by-day plan with contacts, activities and details |
| 6 | Saves shops, travel agencies and tour guides | Stores the selected entries against the user account | Keeps useful local contacts for later in the trip |

## 4.5 Standards and professional practice

The back end follows REST conventions with JSON payloads [42]. Training and inference are separated: no request trains a model, and the bundle is versioned with the code. Secrets are stored in environment variables. Python code follows PEP 8. Requirements are classified using ISO/IEC 25010 [41], and personal data handling follows the Personal Data Protection Act [39]. References follow the IEEE style, and the module's academic-integrity policy is respected through the disclosure in Appendix B.

# 5 IMPLEMENTATION INTEGRATION AND TESTING

This chapter describes how the design was implemented, how the component is integrated with the group platform, how it was tested and how it is deployed and operated.

## 5.1 Development environment and technology stack

**Table 5.1: Technology stack**

| Layer | Technology | Use in the component |
|---|---|---|
| Language and ML | Python 3.10+, scikit-learn [36], pandas, NumPy, joblib | Features, Random Forests, bundle |
| Excel I/O | openpyxl | Reading catalogues, writing the training workbook |
| Back end | Flask, SQLAlchemy | REST routes and services, ORM |
| Database | PostgreSQL | Catalogue copy, trip input, selections |
| Front end | React | Attractions, Accommodation and Itinerary pages |
| Scripts | `RUN_BACKEND.bat`, `RETRAIN_ALL_RF.bat` | Start-up and retraining |
| Tools | Git, Visual Studio Code / Cursor | Version control and development |

## 5.2 Component implementation

The RF package is located at `RF for attraction, accomadation, start time/rf_itinerary/rf_itinerary/` within the group repository. Table 5.2 lists its main modules and the related back-end files.

**Table 5.2: Implementation modules**

| Module | Responsibility |
|---|---|
| `common.py` | `load_attractions()`, `load_hotels()`, time and price parsing, geo lookup, feature builders, physical constants (road factor, average speed) |
| `build_training_data.py` | Synthetic trips, labels and all workbook sheets |
| `train_models.py` | Vectorisers, OOF need predictions, GroupKFold evaluation, bundle export |

| Module | Responsibility |
|---|---|
| rf_planning.py | allocate_destinations(): destination selection, nights, order, day_dest map |
| generate_itinerary.py | generate(): calendar mapping, greedy slot placement with gap, CMB arrival and departure rows, hotel check-in costs |
| rf_itinerary_service.py | Bridge between Flask and the package; bundle_ready() |
| places_rfc_service.py | suggest_from_trip() for attraction suggestions |
| accommodation_rfc_service.py | ranked_hotels_for_destination() and hotel_prices_for_meal() |
| ai_service.py, itinerary_present_service.py | build_itinerary_payload(), presentation enrichment |
| seed_data.py, 008_accommodation_meal_prices.sql | Database seeding and meal-price columns |
| AttractionsPage, AccommodationPage, ItineraryPage | React wizard steps |

### Runtime generation

The listing below summarises the core of generate(). Each commented line corresponds to one model of the bundle; everything else is calendar logic and assembly.

```
days        = (departure.date() - arrival.date()).days + 1
travel_days = days - 2                                  # calendar rule
mood_p, cat_p = need_model.predict_proba(msg_vec(message))  # need
att_p   = att_model.predict_proba(attraction_features(...)) # relevance
day_dest = allocate_destinations(dest_clf, dest_days, dest_order_model, ...)
for day in travel days:
    k = ppd_model.predict(day_features)                     # 2 or 3
    place_greedy(slot_model, day_window, timing_params['gap'], k)
for city in route:
    hotel = argmax(hotel_model.predict_proba(hotel_features(all_hotels)))
```

### Meal-plan pricing

hotel_prices_for_meal() reads the minimum and maximum columns of the selected meal plan. The nightly band is shown as "LKR min – max" and the stay total as "nights × min – nights × max". No percentage uplift or exchange-rate conversion is used in the decision logic; the USD display on the front end is a presentation option only.

### User interface

Figure 5.1 shows the steps of the user interface. After logging in, the traveller enters the trip details, reviews and adjusts the RF-suggested attractions, confirms the RF-ranked hotel for each destination with the meal-plan price band and total, and finally views the itinerary timeline, which can be exported. The saved directory is available from the same account at any time.

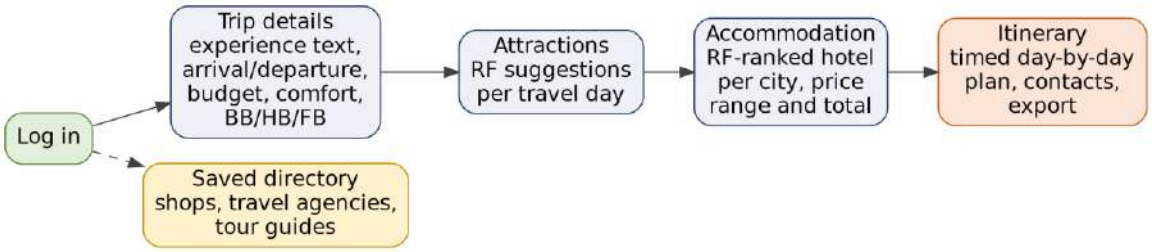



**Figure 5.1: Steps of the itinerary wizard**

### Saved shops, travel agencies and tour guides

Besides the machine-learning pipeline, the component includes a practical feature that helps travellers during the trip: a saved directory. While planning or travelling, the logged-in user can save shops, travel agencies and tour guides to a personal list. Each saved entry keeps its name, type and contact details and is linked to the user's account, so it can be opened again later, for example to contact a guide recommended for a particular city or to return to a shop near the hotel. This feature does not use machine learning; its purpose is to keep useful local contacts in the same place as the itinerary instead of scattered across messaging apps and notes, which is one of the planning problems listed in Table 1.1.

## 5.3 Integration with the other components

The component shares the PostgreSQL tables `Attraction`, `Accommodation` and `UserTripInput` and the destination alias mapping with the platform. It runs stand-alone as long as the bundle is present, and it exposes the trip identifier, dates and selected places through the database so that other services of the group can use them. Figure 5.2 shows the sequence of an itinerary request through the integrated system.

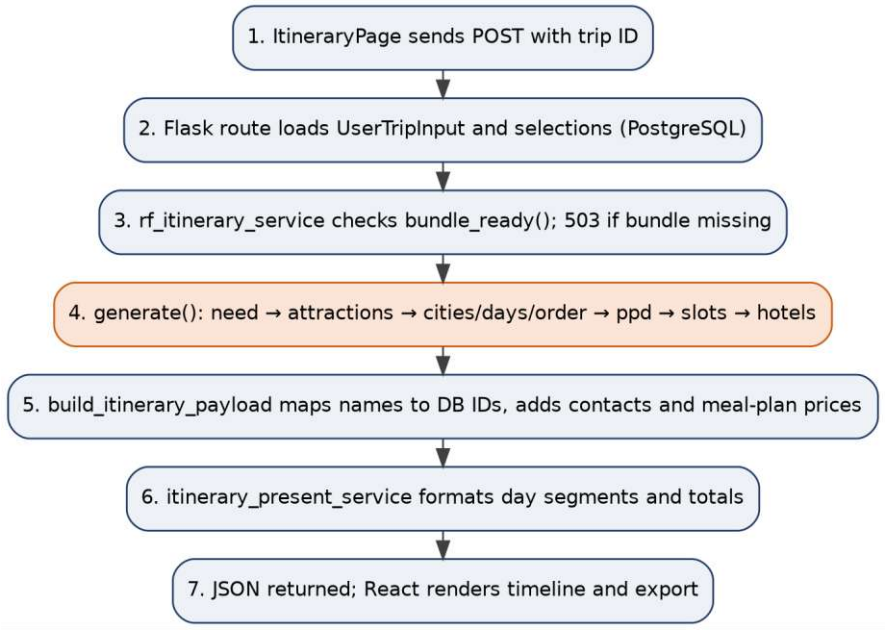



**Figure 5.2: Sequence of an itinerary generation request**

Integration followed the group's shared branch workflow. The main integration issues encountered were destination spelling differences between the catalogue and the database, solved with the alias sheet, and mismatched field names in the trip input, solved by aligning the `UserTripInput` model with the itinerary builder.

### 5.4 Verification and software testing

Verification combined model validation (Chapter 6), API tests and user-interface checks. Table 5.3 lists the main test cases.

**Table 5.3: Functional and integration test cases**

| ID | Test | Expected result | Result |
|---|---|---|---|
| TC1 | POST suggestion request with a nature-and-beach message | Ranked list of catalogue attractions | Pass |
| TC2 | Generate itinerary for a 7-day trip with evening arrival | No visits on day 1; hotel check-in at first city | Pass |
| TC3 | Generate itinerary with morning departure | Last day contains only shopping/rest and departure | Pass |
| TC4 | Inspect visit times on every travel day | No overlaps; learned gap respected | Pass |
| TC5 | Switch meal plan from BB to HB | Hotel price band and total change to HB columns | Pass |

| ID | Test | Expected result | Result |
|---|---|---|---|
| TC6 | Remove `rf_bundle.joblib` and call the itinerary API | HTTP 503 with clear message | Pass |
| TC7 | Change a hotel price in Excel, reseed and retrain | New band visible without code change | Pass |
| TC8 | Log in and save a tour guide and a travel agency | Entries appear in the saved directory after logging in again | Pass |

After each catalogue change, a regression run compared the training log (AUC and F1) with the previous run and regenerated a set of sample itineraries to check that the calendar rules and price displays still held.

### 5.5 Deployment and operation

The component is deployed as part of the group's Flask back end, started with `RUN_BACKEND.bat`, and the React front end. The operating procedure when catalogues change is: (1) update `attractions.xlsx` or `accomadation.xlsx`; (2) run `seed_data.py` to refresh PostgreSQL; (3) run `RETRAIN_ALL_RF.bat` to rebuild the workbook and the bundle; (4) restart Flask so that the new bundle is loaded. For quick checks, the package provides a command-line interface, for example `python generate_itinerary.py --message "beaches and temples" ...`, which prints the itinerary without starting the web application.

# 6 RESULTS AND VALIDATION

This chapter presents the results of training and evaluating the models, the technical results of the integrated product, the stakeholder validation and the integration results. Values marked in the tables come from the final run of `train_models.py` reproduced in Appendix A.

## 6.1 Research results

### Training data

The final run of `build_training_data.py` copied `attractions.xlsx` and `accomadation.xlsx` into the RF package, built the Geo sheet (Bandaranaike International Airport, destination clusters and Local Authority coordinates) and wrote synthetic trips with TF-IDF-based positive and negative attraction labels, destination labels (selection, nights and order), hotel labels for up to 25 nearest hotels per destination, visit-slot alternatives with overlap flags, places-per-day labels and non-overlapping sample itineraries. Table 6.1 summarises the resulting workbook.

**Table 6.1: Size of the training workbook (RF_training_data.xlsx)**

| Sheet | Rows | Role in the itinerary RF |
|---|---|---|
| Geo | 190 | Airport, destinations and Local Authorities for road distance and hotel distance |
| Trips (synthetic travellers) | 171 | user_message, dates, budget, comfort, meal; LABEL_moods, LABEL_categories |
| Train_Attractions | 4,632 | Trip–attraction pairs for att_model (LABEL_relevant) |
| Train_Destinations | 4,446 | Trip–destination pairs for dest_clf, dest_days and dest_order_model |
| Train_Hotels (25 nearest per destination) | 12,625 | Hotel choice labels for hotel_model |
| Train_Visit_Times | 1,610 | Visit-slot alternatives for slot_model (LABEL_chosen) |
| Train_PPD | 403 | Middle-day load for ppd_model (2 or 3 places) |
| Train_Itinerary_Samples | 1,437 | Timed sample rows for timing_params (gap, checkout) |

At training time the catalogues contained 343 attraction–destination records (1,096 timed mood and slot rows in the All Data sheet), 26 destinations and 890 hotels usable after `load_hotels()`.

### Model training results

`train_models.py` trained the TF-IDF vectorisers and the Random Forest models (200 trees each, with `class_weight='balanced'` for the classifiers) and saved them in `rf_bundle.joblib` (about 12 MB). Table 6.2 reports the metrics of the final run.

**Table 6.2: Metrics of the final train_models.py run**

| Model | Task | Metric | Value |
|---|---|---|---|
| need_model | Message → 10 moods + 6 categories | Micro-F1 (5-fold OOF) | 0.793 |
| att_model | Attraction relevance (sim_* + need probabilities) | Mean ROC-AUC (GroupKFold by trip) | 1.000 |
| hotel_model | Hotel choice | Mean ROC-AUC (GroupKFold by trip) | 0.993 |
| dest_clf / dest_days / dest_order_model | Cities, nights, order | Training rows | 4,446 |
| slot_model | Visit start and end slot | Training rows; learned day_window | 1,610; 02:00–23:00 |
| ppd_model | Places per middle day | Training rows | 403 |
| timing_params (not RF) | Gap between visits; checkout offset | Median from samples | 60 min; 120 min |

These results show that the training pipeline produces a workbook of sufficient scale for multi-task Random Forest learning across attraction relevance, routing, visit-time choice and hotel selection. The need model reached a micro-F1 of 0.793, the attraction model a mean ROC-AUC of 1.000 and the hotel model 0.993. Training hotel labels are deliberately restricted to the 25 nearest candidates per destination to keep labelling tractable, while inference in `hotels_for_destination` evaluates every hotel in `accomadation.xlsx`, preserving the requirement of full-registry ranking. The timing medians learned from `Train_Itinerary_Samples` and the chosen visit rows configure the greedy scheduler in `generate_itinerary.py`, which

combines learned probabilities with deterministic overlap and inbound-tour day rules.

The values must nevertheless be interpreted carefully. The attraction labels were derived from TF-IDF similarity between the synthetic message and the attraction text, and the same similarities are features of the attraction model. An AUC of 1.000 therefore shows that the generated data are strongly separable and that the model has learned the labelling rule, not that it would rank attractions perfectly for real travellers. The same applies, to a lesser degree, to the hotel model. Sections 7.1 and 7.5 discuss this possible optimism.

### Comparison with baselines

Table 6.3 compares the RF pipeline qualitatively with the two rule-based baselines from the earlier implementation.

**Table 6.3: RF pipeline compared with rule-based baselines**

| Aspect | Keyword suggestion / even-split plan | RF pipeline |
|---|---|---|
| Synonyms and paraphrases ("waterfalls" vs "falls", "sea" vs "beach") | Missed unless listed | Captured through need probabilities and five field similarities |
| Multi-interest messages | Dominated by the most frequent keyword | Combined through learned weighting |
| Nights per city | Equal split regardless of supply | Predicted from match quality, supply and distance |
| City order | Order of selection | Predicted order from the airport outward |
| Visit times | First listed slot | Slot chosen by mood and rating, no overlap |
| Hotel choice | Short list or highest rating | Full registry scored by price band, comfort, distance and rating |

## 6.2 Product and technical results

The runtime path is `generate_itinerary.generate(message, arrival, departure, budget_lkr, comfort, meal)`, which computes `travel_days = total_calendar_days − 2` and places timed attraction rows only on the middle

days. In the generated output the calendar rules were observed as designed: day 1 contains the *Arrival* event with the CMB time and a *Hotel check-in* when the first city starts; days 2 to $n-1$ contain timed attractions with no same-day overlap and at least the learned 60-minute gap; and day $n$ contains only *Free time / shopping* and *Departure*. In the back end, `suggest_from_trip` and `build_itinerary_payload` use the same bundle, and accommodation ranking applies `hotel_model` to the full catalogue with `hotel_prices_for_meal` for BB, HB or FB.

The end-to-end wizard works as designed: experience text and trip details lead to RF suggestions, RF-ranked hotels with BB, HB or FB price bands, and a complete itinerary with arrival, hotel check-in, timed attractions and departure. Table 6.4 shows an excerpt of an itinerary generated for the message *"I'm on a vacation and want a memorable experience with nature, would love to connect with culture, beaches and nature"*, an 8-day stay, a nightly budget of LKR 30,000, the *Comfortable* level and Half Board.

**Table 6.4: Excerpt of a generated itinerary**

| Day | Event | Details |
|---|---|---|
| 1 | Arrival 19:00 (CMB); hotel check-in | Lipton View Villa, Badulla – 4 nights, HB, LKR 22,500–37,500 per night; stay LKR 90,000–150,000; contacts shown |
| 2 | Dunhida Waterfall access point | 06:00–09:00, Peaceful – forest walk, souvenir stalls, photography |
| 2 | Upper Rawana | 09:30–11:00, Curious – waterfall viewing, hiking |
| 2 | Ravana Waterfall | 16:30–17:30, Relaxed |
| 3–4 | Tea factory tour, Buddha statue, further waterfalls | Morning and late-afternoon slots, no overlaps |
| 5 | Hotel check-in; Matara Beach Park; Matara Bodhiya | Caspian Mirissa, 3 nights, HB; 15:30–17:30 Relaxed; 18:00–18:45 Healing |
| 8 | Free time / shopping; departure 22:30 | No attractions on the departure day |

### The system from the traveller's point of view

From the traveller's side the result is a short, guided process. A visitor who logs in and writes that they want nature, culture and beaches, with an evening arrival and an

eight-day stay, does not have to search for attractions city by city, compare hotel prices for different board types, or check opening times. The system lists attractions that match the request for each travel day, suggests a suitable hotel for each destination within the stated budget and comfort level with the price range for the chosen meal plan, and assigns each visit a start and end time that fits the official visit windows without clashing with other visits. The final day is kept free for shopping and the flight. Useful shops, travel agencies and tour guides can be saved to the same account for later. In this way the component turns an open-ended, multi-tool planning task into a single plan that the traveller can review and follow.

The example shows the intended behaviour. The need model detected *Relaxed*, *Peaceful* and *Curious* as the strongest moods; the destination models chose the Badulla area for nature and waterfalls and the Matara coast for beaches; morning slots were used for peaceful nature visits and late-afternoon slots for relaxed beach visits; and the hotel prices reflect the HB columns of the catalogue. It also shows a weakness: three entries refer to the same Dunhinda waterfall under slightly different names, because the catalogue contains near-duplicates.

### Scenario checks

Multi-city sample itineraries were generated for different trip lengths (4–10 days), all comfort levels and all meal plans, and checked against the rules in Table 6.5.

**Table 6.5: Scenario checks on generated itineraries**

| Check | Result |
|---|---|
| Visits overlapping on the same day | None observed (enforced by the scheduler) |
| Attractions on arrival or departure day | None observed |
| Gap between consecutive visits | At least the learned 60 minutes |
| Hotel prices shown | Taken from the BB, HB or FB columns chosen by the user |
| Entities in the itinerary | Only registered attractions and hotels from the catalogues |

### 6.3 User or stakeholder validation

The supervisor reviewed the calendar rules (arrival day, travel days and departure day) and the presentation of meal-plan prices and confirmed that they match the intended inbound-tour conventions. The co-supervisor's feedback on the evaluation led to the critical discussion of the near-perfect attraction AUC in Section 7.1. Peers who tested the integrated application during development used it with their own travel wishes; their feedback mainly concerned the clarity of the price ranges and the order of the wizard steps, which were refined accordingly. A formal Likert-scale user study with a larger group of real tourists is planned as future work (Section 9.4).

### 6.4 Integration results

Within the integrated platform, the fields of `UserTripInput` are consistent between the trip-input form, the accommodation step and the itinerary builder, so the same trip identifier carries the user's inputs through all steps. The Destination Alias sheet prevents a destination from being split into two entries because of spelling differences. Updating meal prices in `accomadation.xlsx`, reseeding and retraining were sufficient to change the prices shown in both the accommodation page and the itinerary, without code changes.

### 6.5 Summary of findings

- The RF models reproduce the synthetic labelling logic with high cross-validated scores (need micro-F1 0.793, attraction AUC 1.000, hotel AUC 0.993), trained on a workbook of 171 trips and more than 25,000 labelled rows.
- The pipeline improves on keyword suggestion and even-split planning by handling paraphrases, predicting nights and order, choosing mood-appropriate slots and scoring the full hotel registry.
- Generated itineraries respect the overlap and calendar rules and show catalogue-grounded meal-plan prices.
- The whole pipeline is reproducible from the Excel catalogues with one retraining command.

# 7 DISCUSSION

This chapter interprets the results, compares them with the literature and existing solutions, states the contributions and discusses limitations and implications.

## 7.1 Interpretation of findings

The results show that a Random Forest pipeline trained on catalogue-derived labels can drive every decision of a multi-day itinerary. The need model's micro-F1 of 0.793 indicates that moods and categories can be inferred reliably from the kind of messages used in training; this is important because the need probabilities are the strongest link between everyday wording and the catalogue's controlled vocabulary.

The near-perfect attraction AUC deserves a critical reading. Because relevance labels were produced from TF-IDF similarity, and similarity is also a model input, the task the model learns is close to deterministic. This is a known risk when labels and features share a source [31]. The result confirms that the pipeline is technically correct and that the model can rank attractions in the way the label generator intended (supporting H1), but it does not prove that real tourists would agree with the ranking. An independent evaluation with real messages and human relevance judgements is therefore the most important next step.

The hotel model's high AUC under GroupKFold by trip (supporting H2) shows that the combination of meal-plan price relative to budget, comfort–category fit, rating, rating count and distance is learned consistently across trips. Training on the 25 nearest hotels while scoring all hotels at runtime is a deliberate compromise: it keeps the training data small, and the distance feature penalises far-away units at runtime. Because distant units were never labelled, however, their scores are extrapolations.

Regarding RQ3, the separation of concerns worked as intended: the slot model chooses which slot is preferred, while the greedy scheduler guarantees that no two visits overlap. Even if the slot model makes a poor choice, the itinerary remains feasible. The trade-off is that a greedy scheduler cannot revise earlier choices, so some travel days may end with fewer visits than possible. The learned visiting window of 02:00–23:00 is wide because a few catalogue slots start before dawn or

end late; in practice the available slots of each attraction, not the window, determine the visit times.

### 7.2 Comparison with literature baselines and existing solutions

Compared with the keyword baseline, the RF pipeline is more robust to synonyms and multi-interest messages, which agrees with learning-to-rank findings that combined learned signals outperform single hand-tuned scores [34]. Compared with the even-split plan, it allocates nights according to the supply and match quality of each destination.

Compared with orienteering-based planners [17, 18], the component does not search for an optimal route, but it supplies the personalised scores that such planners assume as given; the two approaches are complementary. Compared with personalised recommenders that learn interests from user histories [33], it works without any history, which suits a new platform [22]. Compared with generic LLM chat planners, its main advantage is grounding: every attraction, time and price comes from the registered catalogue. Compared with booking applications, it connects hotels to the attractions and visit windows of the trip.

The choice of Random Forests is consistent with tourism machine-learning studies that applied tree ensembles to tourism data [32] and with benchmarks showing that tree ensembles outperform deep models on medium-sized tabular data [27].

### 7.3 Research contribution

- A multi-model Random Forest bundle that covers need understanding, attraction relevance, destination selection, nights, order, places per day, visit slots and hotels for one domain.
- A dual-Excel weak-supervision pipeline that generates labelled trips from national attraction and accommodation registers.
- A design that combines learned slot preferences with a deterministic overlap-free scheduler and calendar rules.
- A critical analysis of evaluation when labels and features share a source, with a proposal for independent evaluation.

### 7.4 Product contribution

- A working Flask and React module integrated into the group platform, with REST endpoints for suggestion, hotel ranking and itinerary generation.
- Meal-aware hotel economics: nightly BB, HB or FB price bands and stay totals taken directly from the catalogue.
- Operational simplicity: catalogue maintainers update Excel files and run one batch file to retrain.

### 7.5 Limitations and threats to validity

**Synthetic messages.** Training messages are built from catalogue tokens, so their wording is closer to the catalogue than real messages would be. This threatens the construct and external validity of the reported metrics.

**Label–feature overlap.** As discussed in Section 7.1, high AUC values partly reflect the shared TF-IDF source of labels and features.

**Geographic approximation.** Hotel coordinates are approximated at Local Authority level with a geo-hash fallback, and travel time uses straight-line distance × 1.35 at an average speed. Distances and transfer times are therefore estimates.

**Greedy scheduling.** The scheduler does not optimise globally and may leave capacity unused on some days.

**Catalogue quality.** Near-duplicate attractions and the uneven number of attractions per destination (Figure 3.1) affect the itineraries. HB and FB price bands are only as accurate as the data entered.

**Remaining fixed rules.** Table 7.1 lists the parts of the system that are deliberately rule-based.

**Table 7.1: Rule-based parts of the component**

| Rule or parameter | Value / behaviour | Reason |
|---|---|---|
| Arrival day | Airport transfer and hotel only | Business rule |
| Departure day | Shopping, rest and departure only | Business rule |
| Travel days | total_days − 2 | Calendar convention |
| Overlap check | No two visits overlap; learned gap | Feasibility guarantee |

| Rule or parameter | Value / behaviour | Reason |
|---|---|---|
| Road distance and speed | Straight line × 1.35; fixed average speed | No routing data |
| Hotel coordinates | Local Authority approximation, geo-hash fallback | No GPS in catalogue |
| Missing bundle | HTTP 503 | Avoid showing non-ML plans as ML output |

### 7.6 Implications

For tourism informatics, the work shows that national spreadsheets maintained by a tourism authority or platform team can be turned into a personalised planning engine without waiting for years of user logs. For practitioners, it offers a template: keep the catalogue as the single source of truth, generate labels from it, train light models and retrain whenever the data change. For policy, a catalogue-grounded planner can promote registered and less-visited attractions and support the demand-side management of tourist flows discussed in the overtourism literature [5, 6].

# 8 COMMERCIALISATION AND SUSTAINABILITY

This chapter considers how the AI Itinerary Generator could create value beyond the research project.

## 8.1 Value proposition and target users

The primary users are free independent travellers (FITs) who want a fast, transparent and realistic plan without contacting an agent. The generator turns a sentence and a few constraints into a timed plan with registered hotels and clear price bands. Secondary beneficiaries are tour operators and accommodation providers: operators can use generated plans as a starting point for tailored proposals, and registered properties gain visibility when they fit a traveller's budget and comfort level. The saved directory also benefits local shops, travel agencies and tour guides, because travellers keep their details and are more likely to contact them again during the trip.

## 8.2 Market and competitive position

Sri Lanka receives large numbers of international visitors each year [2], and a growing share plan their trips online. Global platforms are strong in reviews and bookings but leave itinerary construction to the traveller, and generic AI planners are not grounded in Sri Lankan registers. The component's competitive position is therefore that of a locally grounded planning engine that can be offered directly to travellers and as a business-to-business planning API for agencies and tourism websites.

## 8.3 Business or sustainability model

A freemium model is proposed. Basic itinerary generation is free for travellers. Premium features such as branded PDF exports, multiple alternative plans and saved trip histories are paid. Agencies pay a subscription for API access and bulk generation. Because the models are light and run on a standard server, operating costs are low, which supports long-term sustainability of the service.

### 8.4 Scalability intellectual property and compliance

The bundle is loaded once and cached, and `generate()` is stateless, so the service can be scaled horizontally behind a load balancer. The catalogue can grow without code changes; only retraining time increases. The intellectual property lies mainly in the methodology, the feature design and the curated training workbook. Compliance requires data minimisation and secure storage of trip inputs under the Personal Data Protection Act [39], and respect for the terms of the sources used to compile the catalogues.

### 8.5 Contribution to the Sustainable Development Goals

The component contributes to the United Nations Sustainable Development Goals [43]. It supports **SDG 8 (Decent work and economic growth)**, target 8.9 on sustainable tourism, by directing visitors to registered attractions and accommodation across many districts; **SDG 9 (Industry, innovation and infrastructure)** by demonstrating an innovative, reproducible digital service for the tourism industry; and **SDG 12 (Responsible consumption and production)**, target 12.b, because the catalogue-driven design makes it possible to promote less-crowded places and off-peak months by changing the data rather than the code.

# 9 CONCLUSION AND FUTURE DIRECTIONS

This chapter concludes the thesis, reviews the achievement of the objectives, restates the contributions and outlines future work.

## 9.1 Conclusion

This research designed, implemented and evaluated an AI Itinerary Generator for Sri Lanka that delivers Random Forest-centred, catalogue-grounded, calendar-valid multi-day plans with meal-aware hotel economics. From six simple inputs, the component infers the traveller's moods and categories, ranks 343 registered attractions, selects destinations, nights and order, schedules non-overlapping visits from official visit windows and chooses one registered hotel per destination with BB, HB or FB price bands. The models are trained offline from synthetic trips generated from the catalogues themselves and are packaged in a single bundle that the Flask services load at runtime. The cross-validated results show that the models learn the intended planning logic, and the integrated product generates coherent itineraries that follow the calendar rules. The main open question is how well the learned rankings match real travellers' preferences, which requires evaluation with real messages.

## 9.2 Achievement of objectives

**Table 9.1: Achievement of objectives**

| Objective | How it was achieved | Status |
|---|---|---|
| O1 Catalogue synchronisation | `seed_data.py`, migration 008, copy to RF package | Achieved |
| O2 Training workbook | `build_training_data.py` → eight sheets, 171 trips | Achieved |
| O3 Model bundle | `train_models.py` → `rf_bundle.joblib` | Achieved |
| O4 Runtime and integration | `generate()`, suggestion, hotel ranking and itinerary services | Achieved |
| O5 Calendar rules | Arrival and departure rules, travel_days = total − 2 | Achieved |
| O6 Evaluation | Micro-F1 0.793, AUC 1.000 / 0.993, API tests, scenario checks | Achieved (real-user evaluation pending) |

| Objective | How it was achieved | Status |
|---|---|---|
| O7 Reproducibility | `RETRAIN_ALL_RF.bat`, documentation | Achieved |

### 9.3 Individual research and product contributions

As described in Section 1.8, the author designed the feature-engineering scheme and the synthetic label logic, implemented the training and runtime scripts, integrated the bundle with the Flask services, added meal-plan pricing to the catalogue and database, and built the front-end flows for attractions, accommodation and the itinerary.

### 9.4 Future directions

- **Real-user evaluation and active learning.** Collect real messages and user edits of generated itineraries and use them as labels, reducing the dependence on synthetic data.
- **Traffic and weather features.** Replace the fixed road factor and speed with routing-service travel times and add weather and seasonality to slot and destination choice.
- **Constraint-programming scheduling.** Use the learned scores in an orienteering or constraint-programming solver with time windows [18] to fill travel days more completely.
- **Multilingual input.** Use multilingual sentence embeddings [24] so that travellers can describe their wishes in other languages.
- **A/B testing.** Compare the RF planner with the keyword baseline in live use, measuring acceptance and edits of suggested items.
- **Catalogue quality.** De-duplicate attractions, add exact coordinates and validate HB and FB prices with properties.

## REFERENCES


[1] Sri Lanka Tourism Development Authority, *Year in Review 2025*. Colombo, Sri Lanka: SLTDA, 2026. [Online]. Available: https://www.sltda.gov.lk/storage/common_media/Year_in_Review_2025_Final_updated_Report_2026_04_02.pdf

[2] Sri Lanka Tourism Development Authority, *Year in Review 2024*. Colombo, Sri Lanka: SLTDA, 2025. [Online]. Available: https://www.sltda.gov.lk/storage/common_media/Year_In_Review_2024_Final_2024_Jan-Dec_updated_20_04_2026.pdf

[3] Sri Lanka Tourism Development Authority, "Statistics." [Online]. Available: https://www.sltda.gov.lk/en/statistics

[4] Central Bank of Sri Lanka, "Earnings from tourism and tourist arrivals," Featured Chart 8, *Annual Economic Review 2025*. Colombo, Sri Lanka: CBSL, 2025. [Online]. Available: https://www.cbsl.gov.lk/sites/default/files/cbslweb_documents/publications/aer/2025/en/25_Featured_Chart_08.pdf

[5] T. Mihalič, "Conceptualising overtourism: A sustainability approach," *Annals of Tourism Research*, vol. 84, 2020.

[6] C. Santos-Rojo, M. Llopis-Amorós, and J. M. García-García, "Overtourism and sustainability: A bibliometric study (2018–2021)," *Technological Forecasting and Social Change*, vol. 188, Art. no. 122285, 2023.

[7] James Lanka, "Sri Lanka travel guide 2025: A practical 10-day itinerary with Colombo, Kandy, Ella and Galle," 2025. [Online]. Available: https://jameslanka.com/sri-lanka-travel-guide-2025-a-practical-10-day-itinerary-with-colombo-kandy-ella-and-galle

[8] Colombo Airport Taxi, "How to plan a Sri Lanka trip." [Online]. Available: https://colomboairporttaxi.info/sri-lanka/How-to-Plan-a-Sri-Lanka-Trip

[9] Silvia's Trips, "Independent travel to Sri Lanka." [Online]. Available: https://silvias-trips.com/independent-travel-to-sri-lanka/

[10] SoloNomads, "Sri Lanka travel cost guide," 2026. [Online]. Available: https://solonomads.com/guides/sri-lanka/cost

[11] WeltFox, "Solo travel in Sri Lanka." [Online]. Available: https://weltfox.com/stories/solo-travel-sri-lanka/

[12] SoloNomads, "Sri Lanka travel guide," 2026. [Online]. Available: https://solonomads.com/guides/sri-lanka

[13] Packzup, "Sri Lanka travel guide." [Online]. Available: https://packzup.com/sri-lanka-travel-guide/

[14] Sri Lanka Spirit, "Sri Lanka is back: Month-by-month travel guide 2024," 2024. [Online]. Available: https://www.srilanka-spirit.com/blog/sri-lanka-is-back-month-by-month-travel-guide-2024/

[15] Sri Lanka Spirit, "Sri Lanka 2025 forecast: What's coming for travelers," 2025. [Online]. Available: https://www.srilanka-spirit.com/blog/sri-lanka-2025-forecast-whats-coming-travelers/

[16] J. Borràs, A. Moreno, and A. Valls, "Intelligent tourism recommender systems: A survey," *Expert Systems with Applications*, vol. 41, no. 16, pp. 7370–7389, 2014.

[17] P. Vansteenwegen, W. Souffriau, G. Vanden Berghe, and D. Van Oudheusden, "The City Trip Planner: An expert system for tourists," *Expert Systems with Applications*, vol. 38, no. 6, pp. 6540–6546, 2011.

[18] A. Gunawan, H. C. Lau, and P. Vansteenwegen, "Orienteering problem: A survey of recent variants, solution approaches and applications," *European Journal of Operational Research*, vol. 255, no. 2, pp. 315–332, 2016.

[19] G. Adomavicius and A. Tuzhilin, "Toward the next generation of recommender systems: A survey of the state-of-the-art and possible extensions," *IEEE Transactions on Knowledge and Data Engineering*, vol. 17, no. 6, pp. 734–749, 2005.

[20] P. Lops, M. de Gemmis, and G. Semeraro, "Content-based recommender systems: State of the art and trends," in *Recommender Systems Handbook*, F. Ricci, L. Rokach, B. Shapira, and P. B. Kantor, Eds. Boston, MA, USA: Springer, 2011, pp. 73–105.

[21] R. Burke, "Hybrid recommender systems: Survey and experiments," *User Modeling and User-Adapted Interaction*, vol. 12, no. 4, pp. 331–370, 2002.

[22] A. I. Schein, A. Popescul, L. H. Ungar, and D. M. Pennock, "Methods and metrics for cold-start recommendations," in *Proc. 25th Annu. Int. ACM SIGIR Conf. Research and Development in Information Retrieval*, 2002, pp. 253–260.

[23] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," *Information Processing & Management*, vol. 24, no. 5, pp. 513–523, 1988.

[24] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. EMNLP-IJCNLP*, 2019, pp. 3982–3992.

[25] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[26] G. Louppe, L. Wehenkel, A. Sutera, and P. Geurts, "Understanding variable importances in forests of randomized trees," in *Advances in Neural Information Processing Systems 26 (NIPS 2013)*, 2013, pp. 431–439.

[27] L. Grinsztajn, E. Oyallon, and G. Varoquaux, "Why do tree-based models still outperform deep learning on typical tabular data?," in *Advances in Neural*

*Information Processing Systems 35 (NeurIPS 2022), Datasets and Benchmarks Track*, 2022.

[28] P. Vansteenwegen, W. Souffriau, and D. Van Oudheusden, "The orienteering problem: A survey," *European Journal of Operational Research*, vol. 209, no. 1, pp. 1–10, 2011.

[29] D. Gavalas, C. Konstantopoulos, K. Mastakas, and G. Pantziou, "A survey on algorithmic approaches for solving tourist trip design problems," *Journal of Heuristics*, vol. 20, no. 3, pp. 291–328, 2014.

[30] A. Ratner, S. H. Bach, H. Ehrenberg, J. Fries, S. Wu, and C. Ré, "Snorkel: Rapid training data creation with weak supervision," *Proceedings of the VLDB Endowment*, vol. 11, no. 3, pp. 269–282, 2017.

[31] S. Kaufman, S. Rosset, C. Perlich, and O. Stitelman, "Leakage in data mining: Formulation, detection, and avoidance," *ACM Transactions on Knowledge Discovery from Data*, vol. 6, no. 4, Art. no. 15, 2012.

[32] J. Bravo, R. Alarcón, C. Valdivia, and O. Serquén, "Application of machine learning techniques to predict visitors to the tourist attractions of the Moche Route in Peru," *Sustainability*, vol. 15, no. 11, Art. no. 8967, 2023.

[33] K. H. Lim, J. Chan, C. Leckie, and S. Karunasekera, "Personalized trip recommendation for tourists based on user interests, points of interest visit durations and visit recency," *Knowledge and Information Systems*, vol. 54, no. 2, pp. 375–406, 2018.

[34] T.-Y. Liu, "Learning to rank for information retrieval," *Foundations and Trends in Information Retrieval*, vol. 3, no. 3, pp. 225–331, 2009.

[35] A. R. Hevner, S. T. March, J. Park, and S. Ram, "Design science in information systems research," *MIS Quarterly*, vol. 28, no. 1, pp. 75–105, 2004.

[36] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[37] T. Fawcett, "An introduction to ROC analysis," *Pattern Recognition Letters*, vol. 27, no. 8, pp. 861–874, 2006.

[38] G. Tsoumakas and I. Katakis, "Multi-label classification: An overview," *International Journal of Data Warehousing and Mining*, vol. 3, no. 3, pp. 1–13, 2007.

[39] *Personal Data Protection Act, No. 9 of 2022*, Parliament of the Democratic Socialist Republic of Sri Lanka, 2022.

[40] U. Gretzel, "Intelligent systems in tourism: A social science perspective," *Annals of Tourism Research*, vol. 38, no. 3, pp. 757–779, 2011.

[41] ISO/IEC 25010:2011, *Systems and Software Engineering — Systems and Software Quality Requirements and Evaluation (SQuaRE) — System and Software Quality Models*. Geneva, Switzerland: ISO, 2011.

[42] R. T. Fielding, "Architectural styles and the design of network-based software architectures," Ph.D. dissertation, Univ. of California, Irvine, CA, USA, 2000.

[43] United Nations General Assembly, "Transforming our world: The 2030 Agenda for Sustainable Development," Resolution A/RES/70/1, Oct. 2015.

# GLOSSARY

**Random Forest (RF)** – An ensemble of decision trees used here for classification and regression inside the bundle.

**TF-IDF** – A text vectorisation method that weights words by their frequency in a text and rarity in the collection.

**rf_bundle.joblib** – The saved file containing the vectorisers, models, timing parameters and day window.

**travel_days** – The number of inclusive trip days minus two (the arrival day and the departure day).

**day_window** – The earliest start and latest end of visits on a travel day, learned from sample itineraries.

**timing_params** – Learned gap between visits and checkout buffer used by the scheduler.

**Anchor POI** – The attraction around which a synthetic training trip and its message are built.

**GroupKFold** – Cross-validation in which all rows of one trip are kept in the same fold.

**BB / HB / FB** – Bed and Breakfast, Half Board and Full Board meal plans, each with its own LKR minimum and maximum in accomadation.xlsx.

**Weak supervision** – Creating training labels from rules or data structure instead of manual annotation.

# APPENDICES

## Appendix A Research instruments or extended results

This appendix describes the structure of the main training sheets of `RF_training_data.xlsx`; the corresponding row counts are given in Table 6.1 and the metrics in Table 6.2.

**Table A.1: Main columns of Train_Attractions**

| Column | Meaning |
|---|---|
| trip_id | Synthetic trip identifier (used for GroupKFold) |
| user_message | Synthetic experience text |
| attraction_name, destination | Catalogue key of the attraction |
| category, mood, activities, details, rating | Catalogue fields |
| LABEL_relevant | Target: 1 if the attraction is relevant to the message |

**Table A.2: Main columns of Train_Hotels**

| Column | Meaning |
|---|---|
| trip_id, destination | Trip and destination of the decision |
| hotel_name, category, rating, rating_count | Catalogue fields |
| distance_km | Approximate distance from the destination |
| user_budget, user_comfort, user_meal_plan | Trip input |
| meal_plan_price_lkr | Min–max band for the chosen meal plan |
| LABEL_selected | Target: 1 for the chosen hotel |

**Table A.3: Main columns of Train_Visit_Times**

| Column | Meaning |
|---|---|
| trip_id, day_no, attraction_name | Scheduling decision |
| visit_start, visit_end, duration_min | Candidate slot from the catalogue |
| mood, rating | Slot mood and rating |

| Column | Meaning |
|---|---|
| already_planned_that_day, overlaps_planned | Context of other visits that day |
| LABEL_chosen | Target: 1 for the slot used |

## Appendix B Academic integrity and AI use disclosure

Generative AI tools were used during this research as listed in Table B.1. All AI-assisted output was reviewed, executed or tested, and edited by the author, who takes full responsibility for the content, code, data and conclusions. Training data are synthetic, and trip text entered by users is stored only with their consent.

**Table B.1: Use of generative AI tools**

| Tool | Purpose | Verification by the author | Sections / artefacts |
|---|---|---|---|
| ChatGPT (OpenAI) | Explanations of programming and machine-learning concepts; debugging suggestions | Suggestions checked against documentation and tested | Back-end services, training scripts |